Epoch-Bound Usage Tokens: Rate-Limited Anonymous Access with Zero-Knowledge Enforcement

Anant Singh

Abstract

We present *Epoch-Bound Usage Tokens (EBUT)*, a privacy-preserving primitive for enforcing rate-limited access in anonymous systems. In settings where users interact without persistent identities, enforcing usage policies such as rate limits and bounded resource consumption is challenging, as traditional approaches rely on identity-based tracking.

EBUT addresses this problem by introducing a token-based mechanism that combines epoch-bound issuance with bounded usage enforcement. Each user derives at most one token per epoch via a deterministic nullifier construction, preventing token accumulation across time and ensuring strict rate limits. Within an epoch, tokens support incremental usage through a recursive spend-and-refresh mechanism, enabling fine-grained control over resource consumption.

The protocol is realized using blind BBS+ signatures, Pedersen commitments, and zero-knowledge proofs, allowing the server to verify compliance with usage policies without learning any information about the user's identity, token linkage, or activity patterns. We formalize the security properties of EBUT and prove that it achieves unforgeability, unlinkability, and rate-limit soundness under standard cryptographic assumptions, including the q-Strong Diffie–Hellman, Discrete Logarithm, and SXDH assumptions in the random oracle model. To achieve practical efficiency, we incorporate a cross-curve batched equality proof that combines BLS12-381 and Ristretto groups, enabling fast Bulletproofs while maintaining rigorous security. We provide a complete, production-ready specification with detailed implementation guidance, API schemas, and performance estimates.

1 Introduction

Privacy-preserving systems aim to enable users to access services without revealing their identity or linking their actions across time. While modern cryptographic constructions provide strong anonymity guarantees, they struggle to enforce usage policies such as rate limits and bounded resource consumption. This tension arises because traditional enforcement mechanisms rely on persistent identities, which inherently violate unlinkability.

This problem becomes particularly acute in systems where resources are costly or constrained. For example, in decentralized storage or subscription-based services, a provider must ensure that each user consumes only a limited amount of resources over time. However, in an anonymous setting, the server cannot distinguish between users or track their behavior across sessions. As a result, a malicious user may accumulate tokens or credentials over time and later consume resources in a burst, violating rate limits and potentially causing service disruption.

Existing anonymous token systems partially address this challenge. Mechanisms such as Anonymous Credit Tokens (ACT) [1] enable users to obtain tokens and spend them in a privacy-preserving manner, supporting features such as partial spending and unlinkable redemption. However, these systems fundamentally treat tokens as accumulable resources: users may obtain tokens over time and spend them arbitrarily later. Consequently, they do not enforce temporal constraints on usage and cannot prevent burst consumption.

A natural approach is to introduce explicit rate-limiting mechanisms, such as per-epoch quotas enforced via zero-knowledge proofs. However, naïve constructions based on accumulators or membership proofs require maintaining dynamic sets and performing complex updates, leading to significant inefficiencies and poor usability. Moreover, such approaches often introduce additional trust assumptions or degrade privacy guarantees.

In this work, we propose a different approach: instead of enforcing rate limits externally, we embed temporal constraints directly into the structure of tokens themselves. We introduce *Epoch-Bound Usage Tokens (EBUT)*, a primitive in which tokens are intrinsically bound to discrete time epochs. Each user can derive at most one token per epoch through a deterministic nullifier construction, ensuring that token issuance is inherently rate-limited and preventing accumulation across time.

Within each epoch, EBUT supports fine-grained usage through a recursive spend-and-refresh mechanism, allowing tokens to be consumed incrementally while preserving unlinkability. All correctness conditions are enforced via zero-knowledge proofs, enabling the server to verify compliance without learning any information about the user's identity or activity patterns.

The key insight of our construction is that temporal structure can be used as a cryptographic enforcement mechanism. By binding tokens to epochs at the protocol level, EBUT eliminates the need for external rate-limiting logic while preserving strong anonymity guarantees.

1.1 Our Contributions

We summarize our main contributions as follows:

- **A new primitive for rate-limited anonymous access.** We introduce *Epoch-Bound Usage Tokens (EBUT)*, a cryptographic primitive that enforces rate limits in anonymous systems. Unlike existing anonymous token mechanisms, EBUT prevents accumulation of tokens across time by binding issuance to discrete epochs.

- **Temporal enforcement via deterministic nullifiers.** We propose a deterministic epoch-based nullifier construction of the form $N_T = k \cdot H_T$, which ensures that each user can derive at most one token per epoch while preserving unlinkability across epochs. This embeds rate-limiting constraints directly into the cryptographic structure of tokens, eliminating the need for external enforcement mechanisms.

- **Non-accumulating token model.** We formalize and realize a token model in which resources cannot be accumulated over time. This directly addresses the burst-consumption problem present in existing anonymous token systems, where users may hoard tokens and later spend them arbitrarily.

- **Recursive intra-epoch usage mechanism.** We design a recursive spend-and-refresh protocol that enables fine-grained resource consumption within a single epoch. This allows tokens to be used incrementally while maintaining strict global rate limits.

- **Formalization of rate-limit soundness.** We introduce and formalize the notion of *rate-limit soundness* for anonymous token systems, capturing the requirement that no adversary can exceed the allowed rate of token generation or usage over time.

- **Efficient zero-knowledge enforcement.** We construct EBUT using blind BBS+ signatures, Pedersen commitments, and zero-knowledge proofs, enabling the server to verify policy compliance without learning any information about the user's identity or activity. The protocol achieves practical efficiency with proof sizes on the order of a few kilobytes and minimal server-side state.

- **Security proofs under standard assumptions.** We prove that EBUT achieves unforgeability, unlinkability, and rate-limit soundness under standard cryptographic assumptions, including the q-Strong Diffie–Hellman assumption, the Discrete Logarithm assumption, and the SXDH assumption in the random oracle model.
- **Cross-curve batched equality proofs.** We provide a complete, self-contained treatment of a dual-curve range proof system that combines BLS12-381 and Ristretto groups, with full security proofs (completeness, knowledge soundness, zero-knowledge). This is of independent interest for protocols requiring efficient range proofs on committed values across different curves.

1.2 Related Work

Anonymous credentials. Anonymous credential systems enable users to authenticate without revealing their identity or linking multiple interactions. Classical constructions, including those based on pairing-based signatures such as BBS+, provide unlinkable credential presentation and selective disclosure [3–5]. These systems focus primarily on identity privacy and attribute verification, but do not directly address enforcement of usage policies such as rate limits or bounded resource consumption.

Anonymous tokens and usage-based systems. Several works extend anonymous credentials to support token-based access and usage control. In particular, Anonymous Credit Tokens (ACT) [1] introduce a model in which users obtain tokens that can be spent incrementally in a privacy-preserving manner. Such systems support partial spending and unlinkable redemption, making them suitable for usage-based billing and resource allocation. However, these constructions fundamentally treat tokens as accumulable resources; a user may obtain tokens over time and later spend them arbitrarily, which allows burst consumption and violates temporal rate limits.

Rate limiting in anonymous systems. Rate limiting in the absence of identities is inherently challenging. Existing approaches often rely on external mechanisms such as per-epoch quotas enforced via accumulators, Merkle trees, or membership proofs combined with zero-knowledge protocols [10]. While these approaches can enforce constraints, they typically require maintaining dynamic state, performing frequent updates, or introducing additional trust assumptions. Moreover, they may incur significant computational overhead and degrade usability. Other approaches use pseudonymous identifiers or linkable tags to enforce limits, but these weaken privacy guarantees by introducing linkability across sessions or epochs.

Temporal enforcement mechanisms. Some systems incorporate time-based validity, such as expiring credentials or time-limited tokens. However, these mechanisms restrict *when* a token can be used, rather than *how frequently* tokens can be generated or consumed. They do not prevent accumulation across time and therefore do not enforce strict rate limits.

Non-transferable anonymous tokens. The recent work of Durak et al. [2] introduces the notion of non-transferable anonymous tokens, where a token is bound to a client's secret key and cannot be transferred without revealing that key. While their focus is on preventing token sharing, our work addresses the orthogonal problem of rate limiting across time. Interestingly, both approaches use a deterministic nullifier to enforce a policy, but in EBUT the nullifier is epoch-based rather than session-based.

Our approach. In contrast to prior work, EBUT embeds temporal constraints directly into the token structure. By binding token issuance to discrete epochs via a deterministic nullifier construction, EBUT ensures that each user can derive at most one token per epoch. This eliminates token accumulation across time while preserving unlinkability. Furthermore, EBUT combines this temporal enforcement with a recursive spend-and-refresh mechanism, enabling fine-grained usage within each epoch without sacrificing privacy. This provides a unified solution to rate limiting in anonymous systems, without requiring external stateful enforcement mechanisms or weakening anonymity guarantees.

2 Preliminaries

We denote by $\lambda \in \mathbb{N}$ the security parameter. For a finite set S, we write $x \xleftarrow{\$} S$ to indicate that x is chosen uniformly at random from S. A function $\varepsilon(\lambda)$ is *negligible* if $\varepsilon(\lambda) = \lambda^{-\omega(1)}$. We use $\mathsf{negl}(\lambda)$ to denote an arbitrary negligible function.

2.1 Bilinear Groups and Cryptographic Assumptions

Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be cyclic groups of prime order q (depending on λ) with generators $g_1 \in \mathbb{G}_1$, $g_2 \in \mathbb{G}_2$. A bilinear map (pairing) is an efficiently computable map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ satisfying:

- Bilinearity: $e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}$ for all $a, b \in \mathbb{Z}_q$.
- Non-degeneracy: $e(g_1, g_2) \neq 1_{\mathbb{G}_T}$.

We assume a Type-3 pairing where no efficiently computable homomorphism exists between $\mathbb{G}_1$ and $\mathbb{G}_2$.

Assumption 2.1 (Discrete Logarithm (DL))**.** Given (g, g^x) where g is a generator of a cyclic group $\mathbb{G}$ of prime order q and $x \xleftarrow{\$} \mathbb{Z}_q$, no probabilistic polynomial-time (PPT) adversary can output x with more than negligible probability.

Assumption 2.2 (q-Strong Diffie–Hellman (q-SDH) [7])**.** Let $\mathbb{G}_1, \mathbb{G}_2$ be groups of prime order q with generators g_1, g_2. Given a $(q+2)$-tuple $(g_1, g_2, g_2^{\gamma}, g_2^{\gamma^2}, \ldots, g_2^{\gamma^q})$ for random $\gamma \xleftarrow{\$} \mathbb{Z}_q^*$, no PPT adversary can output a pair $(x, g_1^{1/(\gamma+x)})$ with $x \in \mathbb{Z}_q \setminus \{-\gamma\}$, except with negligible probability.

Assumption 2.3 (Symmetric eXternal Diffie–Hellman (SXDH))**.** In a Type-3 pairing setting, the Decisional Diffie–Hellman (DDH) assumption holds in both $\mathbb{G}_1$ and $\mathbb{G}_2$. That is, given $(g_1, g_1^a, g_1^b, g_1^c)$, it is hard to decide whether $c = ab$ or random; similarly in $\mathbb{G}_2$.

2.2 BBS+ Signatures

We use the BBS+ signature scheme [5,6] over a bilinear group setting. The scheme allows signing a vector of messages $(m_1, \ldots, m_L) \in \mathbb{Z}_q^L$.

- $\mathsf{Setup}(1^\lambda)$: Output parameters $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, q, g_1, g_2)$ and additional generators $h_0, \ldots, h_L \in \mathbb{G}_1$, derived via a hash-to-curve function with domain separation.
- $\mathsf{KeyGen}()$: Choose $sk \xleftarrow{\$} \mathbb{Z}_q$, set $pk = g_2^{sk}$.

- $\mathsf{Sign}(sk, (m_1, \ldots, m_L))$: Choose random $e, s \xleftarrow{\$} \mathbb{Z}_q$ and compute

$$A = \left(g_1 \cdot h_0^s \cdot \prod_{i=1}^{L} h_i^{m_i} \right)^{\frac{1}{e+sk}} \in \mathbb{G}_1.$$

  Output signature (A, e, s).

- $\mathsf{Verify}(pk, (m_1, \ldots, m_L), (A, e, s))$: Check $A \neq 1_{\mathbb{G}_1}$, $e \neq 0$, and

$$e(A, pk \cdot g_2^e) = e\left(g_1 \cdot h_0^s \cdot \prod_{i=1}^{L} h_i^{m_i}, g_2 \right).$$

BBS+ signatures are existentially unforgeable under chosen-message attacks (EUF-CMA) under the q-SDH assumption.

2.3 Zero-Knowledge Proofs

We employ non-interactive zero-knowledge proofs of knowledge (NIZKPoK) for various relations, constructed via the Fiat–Shamir transform in the random oracle model. Specifically, we use:

- A proof of possession of a BBS+ signature (Protocol 1 in [1]), which allows proving knowledge of a valid signature on a set of hidden messages, with optional selective disclosure.
- Schnorr-style proofs for equality of discrete logarithms across different bases.
- Bulletproofs [8] for range proofs, adapted to be bound to the outer proof context.

All proofs include a domain separator and a context binding to the application to prevent cross-protocol attacks.

2.4 Time Epochs and Deterministic Nullifiers

Time is divided into discrete epochs $T \in \{0, 1, \ldots\}$ of fixed duration. We define a public deterministic hash-to-curve function

$$H_T = \mathcal{H}_{\mathbb{G}_1}(\text{“EBUT:Epoch:”} \parallel T) \in \mathbb{G}_1.$$

For a user secret $k \in \mathbb{Z}_q^*$, the epoch nullifier is $N_T = k \cdot H_T$. Under the SXDH assumption, the values N_T for different epochs are computationally unlinkable.

3 Syntax and Security Definitions

We now formalize the syntax and security requirements for an Epoch-Bound Usage Token (EBUT) scheme. The protocol involves a server $\mathcal{S}$ and multiple clients $\mathcal{C}$. The server is responsible for issuing tokens and verifying their usage; clients request tokens and spend them to access resources. The scheme operates over discrete time epochs $T = 0, 1, 2, \ldots$, and each client is associated with a long-term master secret key.

3.1 Syntax

An EBUT scheme consists of the following polynomial-time algorithms and interactive protocols.

$\mathsf{Setup}(1^\lambda) \to \mathsf{pp}$: On input the security parameter λ, outputs public parameters pp which are common to all parties.

$\mathsf{Server.KeyGen}(\mathsf{pp}) \to (\mathsf{sk}_S, \mathsf{pk}_S)$: Generates the server's long-term key pair.

$\mathsf{Client.KeyGen}(\mathsf{pp}) \to (\mathsf{sk}_C, \mathsf{pk}_C)$: Generates a client's master key pair. The secret key sk_C is a scalar $k_{\mathsf{sub}} \in \mathbb{Z}_q^*$, and the public key pk_C is a group element $g_1^{\mathsf{sk}_C}$. We assume a public-key infrastructure where the server knows the client's public key (or a commitment to it) during issuance.

Issue (interactive): An interactive protocol between a client and the server to obtain a master subscription token.

- $\mathsf{Client.Query}(\mathsf{pp}, \mathsf{sk}_C, \mathsf{pk}_S) \to (\mathsf{query}, \mathsf{st})$: The client produces a request for a master token and local state.
- $\mathsf{Server.Issue}(\mathsf{pp}, \mathsf{sk}_S, \mathsf{pk}_C, \mathsf{query}) \to \mathsf{resp}$: The server verifies the client's request and, if valid, returns a blind signature on the client's secret key and subscription parameters.
- $\mathsf{Client.Final}(\mathsf{st}, \mathsf{resp}) \to (\mathcal{T}_{\mathsf{master}}, \omega)$: The client completes the issuance, obtaining a master token $\mathcal{T}_{\mathsf{master}}$ and a decommitment ω.

The master token $\mathcal{T}_{\mathsf{master}}$ contains a BBS+ signature on $(k_{\mathsf{sub}}, c_{\mathsf{max}}, E_{\mathsf{max}})$, where c_{max} is the maximum credit per epoch and E_{max} is the subscription expiry epoch.

$\mathsf{Refresh}$ (interactive): An interactive protocol executed at the beginning of an epoch $T \leq E_{\mathsf{max}}$ to derive an epoch-bound daily token.

- $\mathsf{Client.Refresh}(\mathsf{pp}, \mathcal{T}_{\mathsf{master}}, T) \to (\mathsf{req}_{\mathsf{ref}}, \mathsf{st})$: The client produces a refresh request including a zero-knowledge proof of a valid master token, the epoch nullifier N_T, and a commitment to a fresh daily secret k_{daily}.
- $\mathsf{Server.VerifyRefresh}(\mathsf{pp}, \mathsf{sk}_S, T, \mathsf{req}_{\mathsf{ref}}) \to \mathsf{resp}_{\mathsf{ref}}/\perp$: The server verifies the proof, checks that the epoch nullifier N_T is fresh (not in $\mathcal{DB}_{\mathsf{epoch}}$), and issues a blind BBS+ signature on $(c_{\mathsf{max}}, k_{\mathsf{daily}}, T)$.
- $\mathsf{Client.FinalRefresh}(\mathsf{st}, \mathsf{resp}_{\mathsf{ref}}) \to \mathcal{T}_{\mathsf{daily}}$: The client finalizes the daily token $\mathcal{T}_{\mathsf{daily}}$ for epoch T.

Spend (interactive): An interactive protocol to spend s credits from a daily token and obtain a refund token for the remaining balance $m = c_{\mathsf{bal}} - s$.

- $\mathsf{Client.Spend}(\mathsf{pp}, \mathcal{T}_{\mathsf{current}}, s) \to (\mathsf{req}_{\mathsf{spend}}, \mathsf{st})$: The client produces a spend request including a zero-knowledge proof of a valid token, the amount s, and a commitment to the new token.
- $\mathsf{Server.VerifySpend}(\mathsf{pp}, \mathsf{sk}_S, s, \mathsf{req}_{\mathsf{spend}}) \to (\mathsf{resp}_{\mathsf{spend}}, \mathsf{out})$: The server verifies the proof, checks that the token's nullifier k_{cur} is fresh (not in $\mathcal{DB}_{\mathsf{spend}}$), and issues a blind refund token. Output $\mathsf{out} = 1$ if accepted, 0 otherwise.
- $\mathsf{Client.FinalSpend}(\mathsf{st}, \mathsf{resp}_{\mathsf{spend}}) \to \mathcal{T}_{\mathsf{new}}$: The client finalizes the new token $\mathcal{T}_{\mathsf{new}}$ with balance m.

The server maintains two databases for double-spending prevention:

- $\mathcal{DB}_{\mathsf{epoch}}$: stores epoch nullifiers $N_T \in \mathbb{G}_1$ observed during $\mathsf{Refresh}$; entries carry a TTL equal to the epoch duration plus the grace period, and are expired individually rather than bulk-cleared at the epoch boundary.
- $\mathcal{DB}_{\mathsf{spend}}$: stores token nullifiers $k_{\mathsf{cur}} \in \mathbb{Z}_q^*$ observed during Spend; entries carry a TTL equal to the epoch duration plus the grace period, and are expired individually rather than bulk-cleared at the epoch boundary.

Additionally, the server may employ an idempotency cache to handle retries gracefully.

3.2 Correctness

An EBUT scheme must satisfy *correctness*: for any honest execution of the protocols, the server accepts and the client obtains valid tokens with the expected balances. Formally, for all λ, all $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, all key pairs $(\mathsf{sk}_S, \mathsf{pk}_S)$, $(\mathsf{sk}_C, \mathsf{pk}_C)$, and all valid parameters $(c_{\mathsf{max}}, E_{\mathsf{max}})$:

- If Issue runs honestly, the client outputs a master token $\mathcal{T}_{\mathsf{master}}$ that verifies under pk_S.
- For any epoch $T \leq E_{\mathsf{max}}$, an honest $\mathsf{Refresh}$ yields a daily token $\mathcal{T}_{\mathsf{daily}}$ with balance c_{max} that is accepted by the server.
- For any spend amount $1 \leq s \leq c_{\mathsf{bal}}$, an honest Spend yields a new token with balance $c_{\mathsf{bal}} - s$ and the server outputs $\mathsf{out} = 1$.

Correctness follows directly from the completeness of the underlying zero-knowledge proofs and the BBS+ signature scheme.

3.3 Security Definitions

We define three core security properties: *unforgeability*, *unlinkability*, and *rate-limit soundness*. The first two are standard for anonymous token systems; the third captures the epoch-bound rate limiting specific to EBUT.

3.3.1 Unforgeability

Unforgeability ensures that an adversary cannot obtain more valid tokens than it is entitled to. We formulate this as a one-more unforgeability game where the adversary can interact with the server via issuance, refresh, and spend oracles. The goal is to produce more tokens (or spend more credits) than the number of master tokens issued.

Game 1 (Unforgeability, UNF)

Let $\mathcal{A}$ be a PPT adversary. The game proceeds as follows:

1. $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $(\mathsf{sk}_S, \mathsf{pk}_S) \leftarrow \mathsf{Server.KeyGen}(\mathsf{pp})$.
2. $\mathcal{A}$ is given pp and pk_S, and has access to the following oracles:
   - $\mathcal{O}_{\mathsf{Issue}}(\mathsf{pk}_C)$: Runs the Issue protocol with $\mathcal{A}$ acting as the client. The server uses pk_C as the client's public key. Increments a counter $q_{\mathsf{issue}}[\mathsf{pk}_C]$.
   - $\mathcal{O}_{\mathsf{Refresh}}(T, \mathsf{pk}_C, \pi)$: $\mathcal{A}$ supplies a refresh request for epoch T and public key pk_C. The server verifies and, if valid, returns a daily token. Increments $q_{\mathsf{refresh}}[\mathsf{pk}_C][T]$.

- $\mathcal{O}_{\mathsf{Spend}}(s, \mathsf{req})$: $\mathcal{A}$ supplies a spend request. The server verifies and, if valid, returns a refund token. Increments q_{spend}.

3. $\mathcal{A}$ outputs a tuple $(\mathsf{pk}_C^*, \mathcal{T}_1, \ldots, \mathcal{T}_k, \omega_1, \ldots, \omega_k)$ and an epoch T^*.

4. $\mathcal{A}$ wins if for the target public key pk_C^*:

   - All tokens $\mathcal{T}_i$ are valid for epoch T^* (or later) and their total credit value exceeds $c_{\mathsf{max}} \cdot q_{\mathsf{issue}}[\mathsf{pk}_C^*]$; *or*
   - The number of valid daily tokens obtained for pk_C^* across all epochs exceeds $q_{\mathsf{issue}}[\mathsf{pk}_C^*]$; *or*
   - There exists an epoch $T \leq E_{\mathsf{max}}$ where the number of valid daily tokens for pk_C^* exceeds 1 (i.e., more than one refresh per epoch).

The advantage of $\mathcal{A}$ is $\mathbf{Adv}^{\mathrm{UNF}}(\mathcal{A}) = \Pr[\mathcal{A} \text{ wins}]$. An EBUT scheme is *unforgeable* if for all PPT $\mathcal{A}$, $\mathbf{Adv}^{\mathrm{UNF}}(\mathcal{A}) \leq \mathrm{negl}(\lambda)$.

Remark. The oracle queries capture both malicious clients and honest-but-curious server interactions. The game directly models the rate-limiting property: even with adaptive issuance queries, the adversary cannot exceed the per-epoch token limit or the total credit bound.

3.3.2 Unlinkability

Unlinkability requires that the server cannot link a refresh or spend transaction to a specific master token or to other transactions of the same client, beyond what is revealed by the public nullifiers (which are unlinkable across epochs).

We define a game where the adversary (acting as a malicious server) chooses two clients and must distinguish which one is participating in a challenge session.

Game 2 (Unlinkability, UNLINK)

Let $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ be a PPT adversary. The game proceeds as follows:

1. $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $(\mathsf{sk}_S, \mathsf{pk}_S) \leftarrow \mathsf{Server.KeyGen}(\mathsf{pp})$. $\mathcal{A}$ is given pp and pk_S.

2. $\mathcal{A}_1$ outputs two client key pairs $(\mathsf{sk}_C^0, \mathsf{pk}_C^0)$ and $(\mathsf{sk}_C^1, \mathsf{pk}_C^1)$, along with state st. Both must be valid.

3. The challenger flips a fair coin $b \xleftarrow{\$} \{0, 1\}$.

4. $\mathcal{A}_2(\mathsf{st})$ interacts with the challenger in two phases:

   - **Issuance phase:** For $i \in \{0, 1\}$, the challenger runs Issue with $\mathcal{A}_2$ acting as the server, using client i's secret key. The adversary learns the master tokens (if any) but not the internal randomness.
   - **Challenge phase:** $\mathcal{A}_2$ selects an epoch T and initiates a $\mathsf{Refresh}$ protocol. The challenger runs the client side using sk_C^b (the challenge client). The adversary observes the refresh request. Similarly, $\mathcal{A}_2$ may request a Spend protocol for a token owned by b.

5. $\mathcal{A}_2$ outputs a guess $b' \in \{0, 1\}$.

$\mathcal{A}$ wins if $b' = b$. The advantage is $\mathbf{Adv}^{\mathrm{UNLINK}}(\mathcal{A}) = |\Pr[b' = b] - 1/2|$. An EBUT scheme is *unlinkable* if for all PPT $\mathcal{A}$, $\mathbf{Adv}^{\mathrm{UNLINK}}(\mathcal{A}) \leq \mathrm{negl}(\lambda)$.

The definition captures that even a malicious server controlling the issuance process cannot link the refresh/spend of a client to its master token or to other epochs.

3.3.3 Rate-Limit Soundness

This new notion formalizes the epoch-bound rate limiting: no adversary can obtain more than one daily token per epoch per master secret, and cannot spend more than c_{max} credits within an epoch. We define a game where the adversary controls both the server and clients, but must respect the token generation limits.

Game 3 (Rate-Limit Soundness, RLS)

Let $\mathcal{A}$ be a PPT adversary. The game proceeds as follows:

1. $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$. $\mathcal{A}$ may generate server keys (possibly maliciously).
2. $\mathcal{A}$ outputs a tuple $(\mathsf{pk}_S, T, \{(N_T^{(i)}, \pi_{\mathsf{refresh}}^{(i)}, \mathcal{T}_{\mathsf{daily}}^{(i)})\}_{i=1}^n)$ where each $\mathcal{T}_{\mathsf{daily}}^{(i)}$ is a daily token for epoch T under pk_S, and $\pi_{\mathsf{refresh}}^{(i)}$ is the corresponding refresh proof.
3. $\mathcal{A}$ wins if:
   - All daily tokens are valid (i.e., $\mathsf{Server.VerifyRefresh}$ accepts each).
   - The epoch nullifiers $N_T^{(i)}$ are all distinct and non-zero.
   - There exist $i \neq j$ such that the underlying master secret k_{sub} is the same (i.e., the tokens were derived from the same master token), yet the adversary obtained $n \geq 2$ daily tokens for the same epoch.

   or the adversary produces a set of spend transactions within epoch T that consume more than c_{max} total credits from a single master token without double-spending the same token nullifier.

The advantage $\mathbf{Adv}^{\mathrm{RLS}}(\mathcal{A}) = \Pr[\mathcal{A} \text{ wins}]$. An EBUT scheme satisfies *rate-limit soundness* if for all PPT $\mathcal{A}$, $\mathbf{Adv}^{\mathrm{RLS}}(\mathcal{A}) \leq \mathrm{negl}(\lambda)$.

Discussion. Rate-limit soundness directly captures the core guarantee of EBUT: it is infeasible to bypass the per-epoch token limit. This is stronger than simple unforgeability because even if the adversary obtains a valid master token, it cannot use it to exceed the epoch quota. The security reduction will rely on the binding properties of the zero-knowledge proofs and the collision-resistance of the nullifier construction.

4 Construction of Epoch-Bound Usage Tokens

In this section we present a concrete instantiation of EBUT using blind BBS+ signatures, Pedersen commitments, and zero-knowledge proofs. The construction relies on a bilinear group setting (Type-3 pairing) and the random oracle model.

4.1 Public Parameters and Key Generation

Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be groups of prime order q with a pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$. Let $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$ be fixed generators.

Public Parameters. The global parameters pp consist of:

- The bilinear group description $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, q, g_1, g_2)$.
- Five independent generators $h_0, h_1, h_2, h_3, h_4 \in \mathbb{G}_1$ for BBS+ signatures and Pedersen commitments. These are derived via a hash-to-curve function $\mathcal{H}_{\mathbb{G}_1}$ with domain separation, e.g., $h_i = \mathcal{H}_{\mathbb{G}_1}(\text{“EBUT:Generator:”} \parallel i)$.
- A context hash H_{ctx} that binds the protocol instance to an application identifier and the public keys, preventing cross-deployment replay. Specifically,

  $$H_{\mathsf{ctx}} = \mathcal{H}_{\mathsf{scalar}}(\text{“EBUT:Context”} \parallel \text{AppID} \parallel \mathsf{pk}_S),$$

  where $\mathcal{H}_{\mathsf{scalar}} : \{0,1\}^* \to \mathbb{Z}_q$ is a hash function modeled as a random oracle. **Implementation requirement:** Because q (the BLS12-381 scalar field order) is a 255-bit prime, a direct reduction of a 256-bit hash output modulo q introduces statistical bias — values in $[0,\ 2^{256} \bmod q)$ are sampled with probability $\approx 2/2^{256}$ while values in $[2^{256} \bmod q,\ q)$ are sampled with probability $\approx 1/2^{256}$, breaking the zero-knowledge property of the proofs. To prevent modulo bias, $\mathcal{H}_{\mathsf{scalar}}$ **must** be implemented by first generating a digest significantly wider than q, then reducing. Concretely, either (a) hash to at least 64 bytes using SHA-512 (or BLAKE2b-512) before reducing modulo q, or (b) use the `expand_message_xmd` function from RFC 9380 with sufficient output length. Raw `SHA-256(data) mod q` is explicitly **prohibited**.

Server Key Generation. The server chooses a secret key $\mathsf{sk}_S = y \xleftarrow{\$} \mathbb{Z}_q$ and sets its public key as $\mathsf{pk}_S = Y = g_2^y \in \mathbb{G}_2$. The same key pair is used for all tokens (master and daily) to simplify the security analysis; alternatively, separate keys could be used.

Client Key Generation. Each client generates a master secret key $k_{\mathsf{sub}} \xleftarrow{\$} \mathbb{Z}_q^*$ (non-zero) and computes the corresponding public key $X = g_1^{k_{\mathsf{sub}}} \in \mathbb{G}_1$. The server is assumed to know X (or a commitment to it) out-of-band during the initial subscription setup.

4.2 Master Subscription Token Issuance

The issuance protocol allows a client to obtain a master token that encodes the subscription limits: maximum credits per epoch $c_{\mathsf{max}} \in [0, 2^{32} - 1]$ and expiry epoch E_{max}. The protocol is blind: the server does not learn the client's secret k_{sub}, only a commitment to it.

1. **Client commitment.** The client chooses randomness $r_{\mathsf{sub}} \xleftarrow{\$} \mathbb{Z}_q$ and computes a Pedersen commitment to k_{sub}:

   $$K_{\mathsf{sub}} = k_{\mathsf{sub}} \cdot h_1 + r_{\mathsf{sub}} \cdot h_0 \in \mathbb{G}_1.$$

   The client sends K_{sub} to the server along with a Schnorr proof of knowledge π_{PoK} of the opening $(k_{\mathsf{sub}}, r_{\mathsf{sub}})$.

   The proof π_{PoK} is a standard Schnorr protocol: the prover picks $\alpha, \beta \xleftarrow{\$} \mathbb{Z}_q$, computes $T = \alpha \cdot h_1 + \beta \cdot h_0$, obtains challenge $c = \mathcal{H}_{\mathsf{scalar}}(H_{\mathsf{ctx}} \parallel K_{\mathsf{sub}} \parallel T \parallel \text{”PoK”})$, and responds with $s_k = \alpha + c \cdot k_{\mathsf{sub}}$, $s_r = \beta + c \cdot r_{\mathsf{sub}}$. The verifier checks $s_k \cdot h_1 + s_r \cdot h_0 \stackrel{?}{=} T + c \cdot K_{\mathsf{sub}}$.

2. **Server signing.** The server knows the subscription parameters c_{max} and E_{max} (which are policy-dependent and can be public). It chooses fresh randomness $e_{\mathsf{sub}}, s'_{\mathsf{sub}} \xleftarrow{\$} \mathbb{Z}_q$ and computes a BBS+ signature on the message vector $(k_{\mathsf{sub}}, c_{\mathsf{max}}, E_{\mathsf{max}})$ with bases (h_1, h_2, h_3):

$$A_{\mathsf{sub}} = \left(g_1 \cdot K_{\mathsf{sub}} \cdot h_0^{s'_{\mathsf{sub}}} \cdot h_2^{c_{\mathsf{max}}} \cdot h_3^{E_{\mathsf{max}}}\right)^{\frac{1}{e_{\mathsf{sub}}+y}} \in \mathbb{G}_1.$$

   The server sends $(A_{\mathsf{sub}}, e_{\mathsf{sub}}, s'_{\mathsf{sub}})$ to the client.

3. **Client finalisation.** The client sets $s_{\mathsf{sub}} = r_{\mathsf{sub}} + s'_{\mathsf{sub}} \bmod q$. The master token is

$$\mathcal{T}_{\mathsf{master}} = (A_{\mathsf{sub}}, e_{\mathsf{sub}}, s_{\mathsf{sub}}, k_{\mathsf{sub}}, c_{\mathsf{max}}, E_{\mathsf{max}}).$$

   The client verifies the signature using the BBS+ verification equation with pk_S.

4.3 Epoch Refresh: Obtaining a Daily Token

At the beginning of epoch $T \leq E_{\mathsf{max}}$, the client derives an epoch-bound daily token. The process uses a zero-knowledge proof that the client holds a valid master token and that the epoch nullifier N_T is correctly computed.

4.3.1 Client-Side Computation

1. **Epoch nullifier.** Compute the epoch base point $H_T = \mathcal{H}_{\mathbb{G}_1}(\text{“EBUT:Epoch:”} \parallel T) \in \mathbb{G}_1$. The epoch nullifier is $N_T = k_{\mathsf{sub}} \cdot H_T \in \mathbb{G}_1$.

2. **Daily token commitment.** Choose a fresh daily secret $k_{\mathsf{daily}} \xleftarrow{\$} \mathbb{Z}_q^*$ and randomness $r_{\mathsf{daily}} \xleftarrow{\$} \mathbb{Z}_q$. Form the commitment:

$$K_{\mathsf{daily}} = c_{\mathsf{max}} \cdot h_4 + k_{\mathsf{daily}} \cdot h_1 + T \cdot h_2 + r_{\mathsf{daily}} \cdot h_0 \in \mathbb{G}_1.$$

   This commitment binds the daily token to epoch T, its credit balance c_{max}, and the fresh secret k_{daily}.

3. **Expiry range proof (Cross-Curve BEQ).** Compute $\Delta = E_{\mathsf{max}} - T$. Choose $r_\Delta \xleftarrow{\$} \mathbb{Z}_{q_{\mathrm{BLS}}}$ and $r_{25519}^{(\Delta)} \xleftarrow{\$} \mathbb{Z}_{q_{25519}}$. Compute BLS12-381 commitment: $C_\Delta = \Delta \cdot h_3 + r_\Delta \cdot h_0$. Compute Ristretto commitment: $C_{25519}^{(\Delta)} = \Delta \cdot G + r_{25519}^{(\Delta)} \cdot H$. Generate the BEQ proof $\pi_{\mathrm{BEQ_exp}}$ for statement $(C_\Delta, C_{25519}^{(\Delta)})$ with auxiliary commitments $(A', \bar{A}, T_{\mathsf{BBS}}, N_T, K_{\mathsf{daily}})$ (the BBS+ proof components are defined below). Define a bridge commitment $C_{\mathsf{bridge}} = C_\Delta + T \cdot h_3 = E_{\mathsf{max}} \cdot h_3 + r_\Delta \cdot h_0$.

4.3.2 Zero-Knowledge Proof π_{refresh}

The client proves knowledge of a valid master token and correct derivation of N_T, K_{daily}, and C_{bridge}. The proof is a variant of the BBS+ zero-knowledge proof of possession (Protocol 1 in [1]) with additional Schnorr equalities.

1. **BBS+ randomization.** Pick $r_1 \xleftarrow{\$} \mathbb{Z}_q^*$ and compute

$$A' = A_{\mathsf{sub}}^{r_1}, \quad \bar{A} = A'^{-e_{\mathsf{sub}}} \cdot \left(g_1 \cdot h_0^{s_{\mathsf{sub}}} \cdot h_1^{k_{\mathsf{sub}}} \cdot h_2^{c_{\mathsf{max}}} \cdot h_3^{E_{\mathsf{max}}}\right)^{r_1}.$$

   Define scaled secrets: $\tilde{k} = k_{\mathsf{sub}} r_1$, $\tilde{c} = c_{\mathsf{max}} r_1$, $\tilde{E} = E_{\mathsf{max}} r_1$, $\tilde{s} = s_{\mathsf{sub}} r_1$.

2. **Commitments.** Choose random blinders $\rho_e, \rho_{r_1}, \rho_{\tilde{s}}, \rho_{\tilde{k}}, \rho_{\tilde{c}}, \rho_{\tilde{E}} \xleftarrow{\$} \mathbb{Z}_q$ and compute

$$T_{\mathsf{BBS}} = A'^{-\rho_e} \cdot g_1^{\rho_{r_1}} \cdot h_0^{\rho_{\tilde{s}}} \cdot h_1^{\rho_{\tilde{k}}} \cdot h_2^{\rho_{\tilde{c}}} \cdot h_3^{\rho_{\tilde{E}}} \in \mathbb{G}_1.$$

Additionally, choose blinders $\rho_u, \rho_v, \rho_w \xleftarrow{\$} \mathbb{Z}_q$ (for $k_{\mathsf{daily}} r_1$, $r_{\mathsf{daily}} r_1$, $r_\Delta r_1$). Compute bridging commitments:

$$\begin{aligned}
T_{\mathsf{scale},N} &= \rho_{r_1} \cdot N_T, & T_N &= \rho_{\tilde{k}} \cdot H_T, \\
T_{\mathsf{scale},D} &= \rho_{r_1} \cdot K_{\mathsf{daily}}, & T_D &= \rho_{\tilde{c}} \cdot h_4 + \rho_u \cdot h_1 + \rho_{r_1} \cdot T \cdot h_2 + \rho_v \cdot h_0, \\
T_{\mathsf{scale},B} &= \rho_{r_1} \cdot C_{\mathsf{bridge}}, & T_B &= \rho_{\tilde{E}} \cdot h_3 + \rho_w \cdot h_0.
\end{aligned}$$

3. **Challenge.** Compute the Fiat–Shamir challenge as

$$c = \mathcal{H}_{\mathsf{scalar}} \left(\begin{array}{c} H_{\mathsf{ctx}} \parallel \mathsf{pk}_S \parallel T \parallel N_T \parallel K_{\mathsf{daily}} \parallel C_\Delta \parallel \pi_{\mathrm{BEQ_exp}} \parallel \\ A' \parallel \bar{A} \parallel T_{\mathsf{BBS}} \parallel T_{\mathsf{scale},N} \parallel T_N \parallel T_{\mathsf{scale},D} \parallel T_D \parallel T_{\mathsf{scale},B} \parallel T_B \parallel \text{“Refresh”} \end{array} \right).$$

4. **Responses.** Compute modulo q:

$$\begin{aligned}
z_e &= \rho_e + c \cdot e_{\mathsf{sub}}, & z_{r_1} &= \rho_{r_1} + c \cdot r_1, \\
z_{\tilde{s}} &= \rho_{\tilde{s}} + c \cdot \tilde{s}, & z_{\tilde{k}} &= \rho_{\tilde{k}} + c \cdot \tilde{k}, \\
z_{\tilde{c}} &= \rho_{\tilde{c}} + c \cdot \tilde{c}, & z_{\tilde{E}} &= \rho_{\tilde{E}} + c \cdot \tilde{E}, \\
z_u &= \rho_u + c \cdot (k_{\mathsf{daily}} r_1), & z_v &= \rho_v + c \cdot (r_{\mathsf{daily}} r_1), \\
z_w &= \rho_w + c \cdot (r_\Delta r_1).
\end{aligned}$$

5. **Output proof.** π_{refresh} consists of:
   - The randomized BBS+ components $A', \bar{A}, T_{\mathsf{BBS}}$.
   - The nullifier bridging commitments $T_{\mathsf{scale},N}, T_N$.
   - The daily token bridging commitments $T_{\mathsf{scale},D}, T_D$.
   - The expiry bridging commitments $T_{\mathsf{scale},B}, T_B$.
   - The scalar responses $z_e, z_{r_1}, z_{\tilde{s}}, z_{\tilde{k}}, z_{\tilde{c}}, z_{\tilde{E}}, z_u, z_v, z_w$.
   - The cross-curve range proof $\pi_{\mathrm{BEQ_exp}}$.

4.3.3 Server Verification

The server performs the following checks in order:

1. **Epoch check:** T must equal the current epoch (or be within a small grace window).

2. **Nullifier freshness check (read-only):** $N_T \notin \mathcal{DB}_{\mathsf{epoch}}$ and $N_T \neq 1_{\mathbb{G}_1}$. **Do not insert N_T at this step.** Inserting before the proofs are verified creates a TOCTOU vulnerability: an adversary can submit a valid N_T with garbage ZK proofs, causing the server to burn the legitimate user's epoch nullifier in step 2 and then abort during proof verification, permanently locking the user out for the entire epoch.

3. **Basic bounds:** $A' \neq 1_{\mathbb{G}_1}$ and $T_{\mathsf{BBS}} \neq 1_{\mathbb{G}_1}$.

4. **Expiry bridge:** $C_{\mathsf{bridge}} \stackrel{?}{=} C_\Delta + T \cdot h_3$.

5. **Recompute challenge** c from the received values (including H_{ctx} and pk_S).

6. **Verify bridging equalities (subtraction form):**

$$z_{\tilde{k}} \cdot H_T - T_N \stackrel{?}{=} z_{r_1} \cdot N_T - T_{\mathsf{scale},N},$$
$$z_{\tilde{c}} \cdot h_4 + z_u \cdot h_1 + z_{r_1} \cdot T \cdot h_2 + z_v \cdot h_0 - T_D \stackrel{?}{=} z_{r_1} \cdot K_{\mathsf{daily}} - T_{\mathsf{scale},D},$$
$$z_{\tilde{E}} \cdot h_3 + z_w \cdot h_0 - T_B \stackrel{?}{=} z_{r_1} \cdot C_{\mathsf{bridge}} - T_{\mathsf{scale},B}.$$

7. **Schnorr multi-scalar multiplication (BBS+ core):**

$$A'^{-z_e} \cdot g_1^{z_{r_1}} \cdot h_0^{z_{\tilde{s}}} \cdot h_1^{z_{\tilde{k}}} \cdot h_2^{z_{\tilde{c}}} \cdot h_3^{z_{\tilde{E}}} \stackrel{?}{=} T_{\mathsf{BBS}} \cdot \bar{A}^c.$$

8. **BEQ verification:** Verify $\pi_{\text{BEQ_exp}}$ with respect to C_Δ.

9. **Pairing check:**

$$e(A', \mathsf{pk}_S) \stackrel{?}{=} e(\bar{A}, g_2).$$

10. **Atomic state mutation (step 10, LAST):** Only after all checks in steps 1–9 pass: atomically insert N_T into $\mathcal{DB}_{\mathsf{epoch}}$ (reject if already present, indicating a concurrent duplicate or replay). This deferred write ensures that a request with a valid N_T but invalid proofs cannot permanently burn the epoch nullifier of a legitimate user.

If all checks pass, the server issues a daily token by choosing $e_{\mathsf{daily}}, s'_{\mathsf{daily}} \stackrel{\$}{\leftarrow} \mathbb{Z}_q$ and computing

$$A_{\mathsf{daily}} = \left(g_1 \cdot K_{\mathsf{daily}} \cdot h_0^{s'_{\mathsf{daily}}} \right)^{\frac{1}{e_{\mathsf{daily}}+y}}.$$

It caches the response for idempotency and sends $(A_{\mathsf{daily}}, e_{\mathsf{daily}}, s'_{\mathsf{daily}})$ to the client.

4.3.4 Client Finalisation

The client sets $s_{\mathsf{daily}} = r_{\mathsf{daily}} + s'_{\mathsf{daily}} \bmod q$. The daily token is

$$\mathcal{T}_{\mathsf{daily}} = (A_{\mathsf{daily}}, e_{\mathsf{daily}}, s_{\mathsf{daily}}, k_{\mathsf{daily}}, c_{\mathsf{max}}, T).$$

4.4 Intra-Epoch Spending

Within epoch T, a client may spend s credits from a current token $\mathcal{T}_{\mathsf{current}} = (A_{\mathsf{cur}}, e_{\mathsf{cur}}, s_{\mathsf{cur}}, k_{\mathsf{cur}}, c_{\mathsf{bal}}, T)$ and obtain a refund token for the remaining balance $m = c_{\mathsf{bal}} - s$. The protocol ensures $1 \le s \le c_{\mathsf{bal}}$ and produces a new token bound to the same epoch T.

4.4.1 Client-Side Computation

1. **Refund commitment.** Choose a fresh secret $k^* \stackrel{\$}{\leftarrow} \mathbb{Z}_q^*$ and randomness $r^* \stackrel{\$}{\leftarrow} \mathbb{Z}_q$. Compute the commitment for the new token:

$$K' = m \cdot h_4 + k^* \cdot h_1 + T \cdot h_2 + r^* \cdot h_0 \in \mathbb{G}_1.$$

2. **Range proof (Cross-Curve BEQ).** Let $m = c_{\mathsf{bal}} - s$. Choose $r_{\mathsf{BP}} \stackrel{\$}{\leftarrow} \mathbb{Z}_{q_{\text{BLS}}}$ and $r_{25519}^{(m)} \stackrel{\$}{\leftarrow} \mathbb{Z}_{q_{25519}}$. Compute BLS12-381 commitment: $C_{\mathsf{BP}} = m \cdot h_4 + r_{\mathsf{BP}} \cdot h_0$. Compute Ristretto commitment: $C_{25519}^{(m)} = m \cdot G + r_{25519}^{(m)} \cdot H$. Generate the BEQ proof $\pi_{\text{BEQ_spend}}$ for statement $(C_{\mathsf{BP}}, C_{25519}^{(m)})$ with auxiliary commitments $(A', \bar{A}, T_{\mathsf{BBS}}, K', C_{\mathsf{total}})$ (the BBS+ proof components are defined below).

3. **Spend bridge.**

$$C_{\mathsf{total}} = K' + s \cdot h_4 = c_{\mathsf{bal}} \cdot h_4 + k^* \cdot h_1 + T \cdot h_2 + r^* \cdot h_0.$$

4.4.2 Zero-Knowledge Proof π_{spend}

The client proves knowledge of a valid daily token on $(c_{\mathsf{bal}}, k_{\mathsf{cur}}, T)$ with k_{cur} and T disclosed, and that $m = c_{\mathsf{bal}} - s$ is correctly formed.

1. **BBS+ randomization.** Pick $r_1 \xleftarrow{\$} \mathbb{Z}_q^*$ and compute

$$A' = A_{\mathsf{cur}}^{r_1}, \quad \bar{A} = A'^{-e_{\mathsf{cur}}} \cdot \left(g_1 \cdot h_0^{s_{\mathsf{cur}}} \cdot h_4^{c_{\mathsf{bal}}} \cdot h_1^{k_{\mathsf{cur}}} \cdot h_2^{T}\right)^{r_1}.$$

   Scaled secrets: $\tilde{c} = c_{\mathsf{bal}} r_1$, $\tilde{s} = s_{\mathsf{cur}} r_1$.

2. **Commitments.** Choose $\rho_e, \rho_{r_1}, \rho_{\tilde{s}}, \rho_{\tilde{c}} \xleftarrow{\$} \mathbb{Z}_q$ and compute

$$T_{\mathsf{BBS}} = A'^{-\rho_e} \cdot g_1^{\rho_{r_1}} \cdot h_0^{\rho_{\tilde{s}}} \cdot h_4^{\rho_{\tilde{c}}} \cdot h_1^{k_{\mathsf{cur}} \cdot \rho_{r_1}} \cdot h_2^{T \cdot \rho_{r_1}}.$$

   Choose additional blinders $\rho_u, \rho_v, \rho_w \xleftarrow{\$} \mathbb{Z}_q$ (for $k^* r_1$, $r^* r_1$, $r_{\mathsf{BP}} r_1$). Define $\rho_m = \rho_{\tilde{c}} - s \cdot \rho_{r_1}$. Compute bridging commitments:

$$\begin{aligned} T_{\mathsf{scale},T} &= \rho_{r_1} \cdot C_{\mathsf{total}}, & T_{\mathsf{total}} &= \rho_{\tilde{c}} \cdot h_4 + \rho_u \cdot h_1 + \rho_{r_1} \cdot T \cdot h_2 + \rho_v \cdot h_0, \\ T_{\mathsf{scale},R} &= \rho_{r_1} \cdot K', & T_{\mathsf{refund}} &= \rho_m \cdot h_4 + \rho_u \cdot h_1 + \rho_{r_1} \cdot T \cdot h_2 + \rho_v \cdot h_0, \\ T_{\mathsf{scale},BP} &= \rho_{r_1} \cdot C_{\mathsf{BP}}, & T_{\mathsf{BP}} &= \rho_m \cdot h_4 + \rho_w \cdot h_0. \end{aligned}$$

3. **Challenge.** The challenge includes a session nonce η for replay prevention:

$$c = \mathcal{H}_{\mathsf{scalar}}\left(\begin{array}{c} H_{\mathsf{ctx}} \parallel \mathsf{pk}_S \parallel s \parallel k_{\mathsf{cur}} \parallel T \parallel \eta \parallel K' \parallel C_{\mathsf{total}} \parallel C_{\mathsf{BP}} \parallel \pi_{\mathrm{BEQ_spend}} \parallel \\ A' \parallel \bar{A} \parallel T_{\mathsf{BBS}} \parallel T_{\mathsf{scale},T} \parallel T_{\mathsf{total}} \parallel T_{\mathsf{scale},R} \parallel T_{\mathsf{refund}} \parallel T_{\mathsf{scale},BP} \parallel T_{\mathsf{BP}} \parallel \text{“Spend”} \end{array} \right).$$

4. **Responses.** Compute:

$$\begin{aligned} z_e &= \rho_e + c \cdot e_{\mathsf{cur}}, & z_{r_1} &= \rho_{r_1} + c \cdot r_1, \\ z_{\tilde{s}} &= \rho_{\tilde{s}} + c \cdot \tilde{s}, & z_{\tilde{c}} &= \rho_{\tilde{c}} + c \cdot \tilde{c}, \\ z_u &= \rho_u + c \cdot (k^* r_1), & z_v &= \rho_v + c \cdot (r^* r_1), \\ z_w &= \rho_w + c \cdot (r_{\mathsf{BP}} r_1). \end{aligned}$$

   Derived response $z_m = z_{\tilde{c}} - s \cdot z_{r_1}$.

5. **Output proof.** π_{spend} consists of:
   - The randomized BBS+ components $A', \bar{A}, T_{\mathsf{BBS}}$.
   - The spend bridging commitments $T_{\mathsf{scale},T}, T_{\mathsf{total}}$.
   - The refund bridging commitments $T_{\mathsf{scale},R}, T_{\mathsf{refund}}$.
   - The range proof bridging commitments $T_{\mathsf{scale},BP}, T_{\mathsf{BP}}$.
   - The scalar responses $z_e, z_{r_1}, z_{\tilde{s}}, z_{\tilde{c}}, z_u, z_v, z_w$.
   - The derived response $z_m = z_{\tilde{c}} - s \cdot z_{r_1}$ (computed by verifier).
   - The cross-curve range proof $\pi_{\mathrm{BEQ_spend}}$.

4.4.3 Server Verification

1. **Amount check:** $1 \leq s \leq 2^{32} - 1$.
2. **Epoch check:** Disclosed T must be current (with grace period).
3. **Idempotency cache check (FIRST):** Compute $h_{\mathsf{id}} = \mathcal{H}_{\mathsf{scalar}}(\text{“EBUT:Idempotency:Spend”} \parallel \mathsf{len}(\pi_{\mathsf{spend}}) \parallel \pi_{\mathsf{spend}} \parallel \eta)$. If h_{id} is in the idempotency cache, return the cached refund token immediately **without checking the double-spend database**.
4. **Nonce read-only check:** Verify that η is non-zero, is exactly 16 bytes, and has not yet been marked spent. In standard mode, check a short-lived cache; in high-privacy mode, verify a Merkle inclusion proof and query the exact nonce store. **Do not mark η as spent at this step.**
5. **Double-spend read-only check:** Check that $k_{\mathsf{cur}} \notin \mathcal{DB}_{\mathsf{spend}}$. **Reject if $k_{\mathsf{cur}} = 0$. Do not insert into $\mathcal{DB}_{\mathsf{spend}}$ at this step.**
6. **Basic bounds:** $A' \neq 1_{\mathbb{G}_1}$, $T_{\mathsf{BBS}} \neq 1_{\mathbb{G}_1}$.
7. **Spend bridge:** $C_{\mathsf{total}} \stackrel{?}{=} K' + s \cdot h_4$.
8. **Recompute** c from received values.
9. **Compute** $z_m = z_{\tilde{c}} - s \cdot z_{r_1}$.
10. **Verify bridging equalities:**

$$z_{\tilde{c}} \cdot h_4 + z_u \cdot h_1 + z_{r_1} \cdot T \cdot h_2 + z_v \cdot h_0 - T_{\mathsf{total}} \stackrel{?}{=} z_{r_1} \cdot C_{\mathsf{total}} - T_{\mathsf{scale},T},$$
$$z_m \cdot h_4 + z_u \cdot h_1 + z_{r_1} \cdot T \cdot h_2 + z_v \cdot h_0 - T_{\mathsf{refund}} \stackrel{?}{=} z_{r_1} \cdot K' - T_{\mathsf{scale},R},$$
$$z_m \cdot h_4 + z_w \cdot h_0 - T_{\mathsf{BP}} \stackrel{?}{=} z_{r_1} \cdot C_{\mathsf{BP}} - T_{\mathsf{scale},BP}.$$

11. **Schnorr MSM:**
$$A'^{-z_e} \cdot g_1^{z_{r_1}} \cdot h_0^{z_{\tilde{s}}} \cdot h_4^{z_{\tilde{c}}} \cdot h_1^{k_{\mathsf{cur}} \cdot z_{r_1}} \cdot h_2^{T \cdot z_{r_1}} \stackrel{?}{=} T_{\mathsf{BBS}} \cdot \bar{A}^c.$$
12. **BEQ verification:** Verify $\pi_{\text{BEQ_spend}}$ w.r.t. C_{BP}.
13. **Pairing check:** $e(A', \mathsf{pk}_S) = e(\bar{A}, g_2)$.
14. **Atomic state mutation (step 14, LAST):** Only after all checks in steps 1–13 pass: atomically insert k_{cur} into $\mathcal{DB}_{\mathsf{spend}}$ (reject if already present, indicating a concurrent double-spend race); atomically mark η as spent in the nonce store. These writes must be performed as a single atomic transaction or with optimistic concurrency control to prevent TOCTOU races.

If all checks pass and state mutations succeed, the server issues a refund token by choosing $e^*, s'^* \stackrel{\$}{\leftarrow} \mathbb{Z}_q$ and computing

$$A^* = \left(g_1 \cdot K' \cdot h_0^{s'^*}\right)^{\frac{1}{e^*+y}}.$$

It caches the response for idempotency and sends (A^*, e^*, s'^*) to the client.

4.4.4 Client Finalisation

The client sets $s^* = r^* + s'^* \bmod q$ and obtains the new token

$$\mathcal{T}_{\mathsf{new}} = (A^*, e^*, s^*, k^*, m, T).$$

5 Cross-Curve Batched Equality Proofs for Range Arguments

The EBUT protocol requires efficient zero-knowledge range proofs for 32-bit values (balances and expiry deltas). While Bulletproofs over BLS12-381 are sound, their performance on the BLS12-381 scalar field is suboptimal due to the lack of assembly-optimized inner-product arguments. Conversely, the `curve25519-dalek` library provides extremely fast, constant-time Bulletproofs for the Ristretto group.

To combine the security of BLS12-381 signatures with the efficiency of Dalek Bulletproofs, we employ a *cross-curve batched equality proof* (BEQ). This construction proves that a single integer $m \in [0, 2^{32} - 1]$ is simultaneously committed in a BLS12-381 Pedersen commitment and a Ristretto Pedersen commitment, and that m lies in the required range. The proof consists of:

- A *Sigma protocol over the integers* that establishes equality of the committed values across the two curves, and
- A *Dalek Bulletproof* on the Ristretto commitment that enforces $m \in [0, 2^{32} - 1]$.

Because the two curves have distinct scalar field orders ($q_{\text{BLS}} \approx 2^{255}$ and $q_{25519} \approx 2^{252}$), care must be taken to ensure that the integer responses of the Sigma protocol are valid in both groups simultaneously. We achieve this by working directly over the integers and by enforcing a strict bound on the size of the response.

5.1 Notation and Preliminaries

$\mathbb{G}_{\text{BLS}}$	BLS12-381 G1 group (written additively).
$\mathbb{G}_{25519}$	Ristretto group over Curve25519 (written additively).
q_{BLS}	Prime order of $\mathbb{G}_{\text{BLS}}$ ($\approx 2^{255}$).
q_{25519}	Prime order of $\mathbb{G}_{25519}$ ($\approx 2^{252}$).
$h_4, h_0 \in \mathbb{G}_{\text{BLS}}$	Fixed generators for value and blinding in BLS12-381 commitments.
$G, H \in \mathbb{G}_{25519}$	Standard Ristretto Pedersen generators.
$\mathcal{H}_{128}$	Hash function $\{0,1\}^* \to \{0,1\}^{128}$ (for cross-curve challenge).

We rely on the *Discrete Logarithm (DL)* assumption in both groups and on the random oracle model for hash functions. Because the two groups are defined over different fields, we assume there is no efficiently computable homomorphism between them; this is guaranteed by the distinct curve parameters.

5.2 Integer Arithmetic and Bounds

All Sigma protocol operations involving the secret value m are performed over the integers $\mathbb{Z}$ rather than modulo a group order. We define the following bounds:

- $m \in [0, 2^{32} - 1]$ (enforced by the Bulletproof).
- The Fiat–Shamir challenge c is a 128-bit integer, i.e. $c \in [0, 2^{128} - 1]$.

- The blinding integer u_m is drawn uniformly from $[0, 2^{240} - 1]$.

The response $z = u_m + c \cdot m$ therefore satisfies $0 \le z < 2^{240} + 2^{128} \cdot (2^{32} - 1) < 2^{241}$. Crucially, $2^{241} < q_{25519} < q_{\mathrm{BLS}}$, so z is strictly smaller than the order of both groups. Consequently, interpreting z as a scalar modulo q_{BLS} or q_{25519} is unambiguous: no modular reduction ever occurs, and the integer value of z is preserved exactly in both groups.

5.3 The Batched Equality Proof Protocol

The protocol proves knowledge of $(m, r_{\mathrm{BLS}}, r_{25519})$ such that:

$$\begin{aligned} C_{\mathrm{BLS}} &= m \cdot h_4 + r_{\mathrm{BLS}} \cdot h_0 \quad &\in \mathbb{G}_{\mathrm{BLS}}, \\ C_{25519} &= m \cdot G + r_{25519} \cdot H \quad &\in \mathbb{G}_{25519}, \end{aligned}$$

and that $m \in [0, 2^{32} - 1]$.

5.3.1 Prover Algorithm

Input: $m \in [0, 2^{32} - 1]$, $r_{\mathrm{BLS}} \in \mathbb{Z}_{q_{\mathrm{BLS}}}$, $r_{25519} \in \mathbb{Z}_{q_{25519}}$, context string ctx, and optional auxiliary commitments $\mathrm{comm}_1, \ldots, \mathrm{comm}_k \in \mathbb{G}_{\mathrm{BLS}}$ (e.g., BBS+ proof components).

1. **Commitments (with rejection sampling):** Repeat the following until acceptance:
   (a) Choose random integers
   $$u_m \xleftarrow{\$} [0, 2^{240} - 1], \qquad u_{\mathrm{BLS}} \xleftarrow{\$} \mathbb{Z}_{q_{\mathrm{BLS}}}, \qquad u_{25519} \xleftarrow{\$} \mathbb{Z}_{q_{25519}}.$$
   (b) Compute
   $$R_{\mathrm{BLS}} = u_m \cdot h_4 + u_{\mathrm{BLS}} \cdot h_0 \in \mathbb{G}_{\mathrm{BLS}}, \qquad R_{25519} = u_m \cdot G + u_{25519} \cdot H \in \mathbb{G}_{25519}.$$
   (c) Compute the challenge c as in step 3 below.
   (d) Set $z = u_m + c \cdot m$ (as an integer).
   (e) **Rejection test:** if $z > 2^{240} - 1$, **abort and restart** from the beginning of this loop with fresh randomness.
   (f) Otherwise **accept** and continue.

   The expected number of iterations is less than 2; rejection occurs with probability at most $(2^{32} - 1) \cdot (2^{128} - 1)/2^{240} < 2^{-80}$, where $c_{\max} \le 2^{32} - 1$ per Section 4.1 and $c < 2^{128}$.

2. **Range Proof:** Generate a Dalek Bulletproof π_{range} for the Ristretto commitment C_{25519} proving that $m \in [0, 2^{32} - 1]$. The Bulletproof transcript is initialized with domain separator `"EBUT:BEQ:Range"` and includes ctx and the serialized auxiliary commitments comm_i.

3. **Challenge:** Compute the 128-bit challenge
   $$c = \mathcal{H}_{128}\big(\text{“EBUT:BEQ:Challenge”} \parallel \mathrm{ctx} \parallel C_{\mathrm{BLS}} \parallel C_{25519} \parallel R_{\mathrm{BLS}} \parallel R_{25519} \parallel \pi_{\mathrm{range}} \parallel \mathrm{comm}_1 \parallel \cdots \parallel \mathrm{comm}_k\big).$$

4. **Responses:** Use the $z = u_m + c \cdot m$ accepted in step 1 (over the integers, not reduced modulo any group order). Compute modular responses:
   $$z_{\mathrm{BLS}} = u_{\mathrm{BLS}} + c \cdot r_{\mathrm{BLS}} \pmod{q_{\mathrm{BLS}}}, \qquad z_{25519} = u_{25519} + c \cdot r_{25519} \pmod{q_{25519}}.$$
   Here c is interpreted as an element of the respective scalar fields by reducing modulo q_{BLS} and q_{25519} respectively; because $c < 2^{128} \ll \min(q_{\mathrm{BLS}}, q_{25519})$, this reduction is the identity map.

5. **Output:** The proof $\pi_{\mathrm{BEQ}} = (C_{25519}, R_{\mathrm{BLS}}, R_{25519}, z, z_{\mathrm{BLS}}, z_{25519}, \pi_{\mathrm{range}})$.

5.3.2 Verifier Algorithm

Input: π_{BEQ}, C_{BLS}, context ctx, auxiliary commitments $\text{comm}_1, \ldots, \text{comm}_k$.

1. **Bound Check:** Verify that $0 \leq z \leq 2^{240} - 1$ (as an integer). If not, reject. This tighter bound (compared to checking only $z < q_{25519}$, where $q_{25519} \approx 2^{252}$) is enforced by the prover's rejection-sampling step and is required for the zero-knowledge guarantee: without it, a large $z > 2^{240} - 1$ would reveal that $c \cdot m$ is large, leaking a lower bound on the secret m.

2. **Recompute Challenge:** Compute $c = \mathcal{H}_{128}(\text{“EBUT:BEQ:Challenge”} \parallel \text{ctx} \parallel C_{\text{BLS}} \parallel C_{25519} \parallel R_{\text{BLS}} \parallel R_{25519} \parallel \pi_{\text{range}} \parallel \text{comm}_1 \parallel \ldots)$.

3. **Verify BLS12-381 Equation:**

$$z \cdot h_4 + z_{\text{BLS}} \cdot h_0 \stackrel{?}{=} R_{\text{BLS}} + c \cdot C_{\text{BLS}} \quad \text{in } \mathbb{G}_{\text{BLS}}.$$

4. **Verify Ristretto Equation:**

$$z \cdot G + z_{25519} \cdot H \stackrel{?}{=} R_{25519} + c \cdot C_{25519} \quad \text{in } \mathbb{G}_{25519}.$$

5. **Verify Bulletproof:** Verify π_{range} with respect to C_{25519} using the same transcript initialization as the prover.

If all checks pass, the verifier accepts.

5.4 Security Analysis of BEQ

Theorem 5.1 (Completeness)**.** If the prover follows the protocol honestly, the verifier always accepts.

Proof. The rejection-sampling loop guarantees that the accepted $z = u_m + c \cdot m$ satisfies $0 \leq z \leq 2^{240} - 1 < q_{25519}$ (where $q_{25519} = 2^{252} + 27742317777372353535851937790883648493$), so the verifier's bound check passes. Substituting the definitions:

$$z \cdot h_4 + z_{\text{BLS}} \cdot h_0 = (u_m + cm)h_4 + (u_{\text{BLS}} + cr_{\text{BLS}})h_0 = R_{\text{BLS}} + c \cdot C_{\text{BLS}},$$

and similarly for the Ristretto equation. The Bulletproof is complete by the underlying Dalek implementation. Thus, all checks succeed. □

Theorem 5.2 (Knowledge Soundness)**.** Assume the discrete logarithm problem is hard in both $\mathbb{G}_{\text{BLS}}$ and $\mathbb{G}_{25519}$, and that the Bulletproof is computationally sound. Let $\mathcal{A}$ be a PPT adversary making at most q_{RO} queries to $\mathcal{H}_{128}$ and producing a valid proof π_{BEQ} with probability ϵ. Then there exists an extractor $\mathcal{E}$ that, given black-box access to $\mathcal{A}$, outputs a witness $(m, r_{\text{BLS}}, r_{25519})$ such that $m \in [0, 2^{32} - 1]$ and the commitment equations hold, with success probability at least

$$\frac{\epsilon^2}{q_{\text{RO}}} - \frac{\epsilon}{2^{128}} - \mathbf{Adv}^{\text{DL}}(\mathcal{B}) - \mathbf{Adv}^{\text{BP-sound}}(\mathcal{B}').$$

Proof. Consider the interactive version of the protocol where the challenge c is chosen uniformly from $\{0,1\}^{128}$ by the verifier. By the general forking lemma [9] with challenge space size $|\mathcal{C}| = 2^{128}$, if $\mathcal{A}$ succeeds with probability ϵ, then with probability at least $\epsilon^2/q_{\mathrm{RO}} - \epsilon/2^{128}$ we can rewind $\mathcal{A}$ to obtain two accepting transcripts with the same first message $(C_{25519}, R_{\mathrm{BLS}}, R_{25519}, \pi_{\mathrm{range}})$ and distinct challenges $c \neq c'$. Let the responses be $(z, z_{\mathrm{BLS}}, z_{25519})$ and $(z', z'_{\mathrm{BLS}}, z'_{25519})$. From the verification equations we have:

$$(z - z') \cdot h_4 + (z_{\mathrm{BLS}} - z'_{\mathrm{BLS}}) \cdot h_0 = (c - c') \cdot C_{\mathrm{BLS}} \quad \text{in } \mathbb{G}_{\mathrm{BLS}},$$
$$(z - z') \cdot G + (z_{25519} - z'_{25519}) \cdot H = (c - c') \cdot C_{25519} \quad \text{in } \mathbb{G}_{25519}.$$

Define $\Delta z = z - z'$, $\Delta z_{\mathrm{BLS}} = z_{\mathrm{BLS}} - z'_{\mathrm{BLS}}$, $\Delta z_{25519} = z_{25519} - z'_{25519}$, and $\Delta c = c - c'$. Since $c \neq c'$, $\Delta c \neq 0$ and $|\Delta c| < 2^{128}$. The verifier's bound check ensures $0 \leq z, z' < q_{25519}$, so $|\Delta z| < q_{25519}$. Because $q_{25519} \approx 2^{252}$, the integer division $\Delta z / \Delta c$ yields a unique integer m with $|m| < q_{25519}$. Moreover, since $z \geq 0$ and $c \geq 0$, we can show $m \geq 0$ by considering the two cases of Δc sign.

Now consider the first equation in $\mathbb{G}_{\mathrm{BLS}}$. Since $|\Delta z| < q_{\mathrm{BLS}}$, the equality holds over integers when interpreted as scalars modulo q_{BLS}. Thus we can write

$$C_{\mathrm{BLS}} = \left(\frac{\Delta z}{\Delta c}\right) \cdot h_4 + \left(\frac{\Delta z_{\mathrm{BLS}}}{\Delta c}\right) \cdot h_0 \pmod{q_{\mathrm{BLS}}}.$$

Set $m = \frac{\Delta z}{\Delta c}$ (computed over integers) and $r_{\mathrm{BLS}} = \frac{\Delta z_{\mathrm{BLS}}}{\Delta c} \bmod q_{\mathrm{BLS}}$. Similarly, from the second equation we obtain m and $r_{25519} = \frac{\Delta z_{25519}}{\Delta c} \bmod q_{25519}$. The Bulletproof soundness guarantees $m \in [0, 2^{32} - 1]$ except with probability $\mathbf{Adv}^{\text{BP-sound}}(\mathcal{B}')$. If the adversary produces proofs that verify but extraction fails, we can embed a DL challenge into the generators and extract a discrete logarithm from the discrepancy; the reduction is standard and omitted. The bound follows. □

Theorem 5.3 (Zero-Knowledge)**.** Under the random oracle model, the non-interactive BEQ protocol with rejection sampling is statistically zero-knowledge with distinguishing advantage bounded by 2^{-80}. Moreover, the support of z in real executions is exactly $[0, 2^{240} - 1]$—the same as the simulator—so no transcript component is outside the simulator's range.

Proof. We construct a simulator $\mathcal{S}$ that, on input $(C_{\mathrm{BLS}}, \mathrm{ctx})$ and without knowledge of the witness, produces a transcript $\pi_{\mathrm{BEQ}} = (C_{25519}, R_{\mathrm{BLS}}, R_{25519}, z, z_{\mathrm{BLS}}, z_{25519}, \pi_{\mathrm{range}})$ as follows:

1. Choose a random challenge $c \xleftarrow{\$} \{0,1\}^{128}$.
2. Choose random $z \xleftarrow{\$} [0, 2^{240} - 1]$, $z_{\mathrm{BLS}} \xleftarrow{\$} \mathbb{Z}_{q_{\mathrm{BLS}}}$, $z_{25519} \xleftarrow{\$} \mathbb{Z}_{q_{25519}}$.
3. Compute $R_{\mathrm{BLS}} = z \cdot h_4 + z_{\mathrm{BLS}} \cdot h_0 - c \cdot C_{\mathrm{BLS}}$ and $R_{25519} = z \cdot G + z_{25519} \cdot H - c \cdot C_{25519}$, where C_{25519} is an arbitrary commitment to 0.
4. Generate a simulated Bulletproof π_{range} for C_{25519} with range $[0, 2^{32} - 1]$ using the honest-verifier zero-knowledge property of Dalek Bulletproofs (by programming the internal random oracle).
5. Program $\mathcal{H}_{128}$ to return c on the appropriate input.

In a real execution with rejection sampling, the prover discards any u_m for which $z = u_m + c \cdot m > 2^{240} - 1$. For any fixed challenge c and message m, the map $u_m \mapsto z = u_m + c \cdot m$ is a bijection on integers, so conditioning on $z \leq 2^{240} - 1$ is equivalent to conditioning on $u_m \leq 2^{240} - 1 - c \cdot m$. Since u_m is uniform over $[0, 2^{240} - 1]$ and the accepted values fill a contiguous sub-interval of length

$2^{240} - c \cdot m$, the accepted z is uniform over $[0, 2^{240} - 1 - c \cdot m] \subseteq [0, 2^{240} - 1]$. The simulated z is uniform over the full $[0, 2^{240} - 1]$. The statistical distance between these distributions is $c \cdot m/2^{240} \leq (2^{128} - 1)(2^{32} - 1)/2^{240} < 2^{-80}$, which is negligible—but more importantly, both distributions are supported on $[0, 2^{240} - 1]$, so no single value of z is impossible in simulation yet possible in real execution or vice versa. The other components are identically distributed. Hence, the simulation achieves negligible statistical distance, and the distinguishing advantage is at most 2^{-80}. □

5.4.1 Transcript Binding

To prevent splicing attacks where a Bulletproof generated for one context is reused in another, the prover **must** include the BBS+ commitments (A', $\bar{A}$, T_{BBS}) and any bridge commitments in the Bulletproof transcript. Concretely, the transcript is initialized as:

```
Transcript::new(b"EBUT:BEQ:Range");
transcript.append_message(b"ctx", context);
for comm in auxiliary_commitments {
    transcript.append_message(b"comm", comm.to_compressed());
}
```

This binds the range proof to the specific BBS+ proof instance, as required by the EBUT construction.

5.5 Integration into EBUT

The cross-curve batched equality proof replaces all standalone range proofs in the EBUT protocol.

- **Epoch Refresh:** The commitment C_Δ to the expiry delta $\Delta = E_{\mathsf{max}} - T$ is bound to a Ristretto commitment and a range proof via $\pi_{\text{BEQ_exp}}$. The auxiliary commitments include $A', \bar{A}, T_{\mathsf{BBS}}, N_T, K_{\mathsf{daily}}$.
- **Spend Phase:** The commitment C_{BP} to the remaining balance $m = c_{\mathsf{bal}} - s$ is linked to a Ristretto commitment via $\pi_{\text{BEQ_spend}}$. Auxiliary commitments include $A', \bar{A}, T_{\mathsf{BBS}}, K', C_{\mathsf{total}}$.

The proof size is approximately 672 bytes for the Bulletproof (single 32-bit range) plus the Sigma protocol responses ($\approx$ 160 bytes), totaling under 1 KB. Verification time is dominated by the Bulletproof ($\approx$ 1–2 ms) and the two multi-scalar multiplications, which are negligible by comparison.

6 Security Analysis of EBUT

We now prove that the EBUT construction satisfies the three security properties defined in Section 3: unforgeability, unlinkability, and rate-limit soundness. All proofs are in the random oracle model and rely on the hardness assumptions stated in Section 2. We assume the cross-curve BEQ is used for range proofs; its security properties have been established in Section 5.

6.1 Unforgeability

Theorem 6.1 (Unforgeability)**.** Let $\mathcal{A}$ be a PPT adversary against the UNF game making at most q_I issuance queries, q_R refresh queries, q_S spend queries, and q_{RO} random oracle queries. Then

there exist PPT adversaries $\mathcal{B}_{\mathrm{SDH}}$ and $\mathcal{B}_{\mathrm{DL}}$ such that

$$\mathbf{Adv}^{\mathrm{UNF}}(\mathcal{A}) \leq \mathbf{Adv}^{\text{q-SDH}}(\mathcal{B}_{\mathrm{SDH}}) + \mathbf{Adv}^{\mathrm{DL}}(\mathcal{B}_{\mathrm{DL}}) + \frac{(q_I + q_R + q_S + q_{\mathrm{RO}})^2}{2q} + \frac{q_R + q_S}{q} + \frac{1}{2^{128}} + \mathrm{negl}(\lambda),$$

where q is the group order.

Proof. We proceed through a sequence of games $\mathsf{G}_0, \ldots, \mathsf{G}_8$. Let $\Pr[S_i]$ denote the probability that $\mathcal{A}$ wins in Game G_i.

Game G_0: The original UNF game. The challenger $\mathcal{C}$ runs Setup, generates $(\mathsf{sk}_S, \mathsf{pk}_S) = (y, g_2^y)$, and answers all oracles honestly.

Game G_1 (Simulation of zero-knowledge proofs): $\mathcal{C}$ replaces all zero-knowledge proofs (π_{refresh}, π_{spend}, and the internal BEQ proofs) with simulated proofs using the simulators from the ZK theorems. The simulation is perfect under DL and the statistical ZK of BEQ. Any distinguisher gives an adversary against DL or the statistical test. Thus,

$$|\Pr[S_1] - \Pr[S_0]| \leq \mathbf{Adv}^{\mathrm{DL}}(\mathcal{B}_1) + \frac{q_{\mathrm{RO}}^2}{2q} + 2^{-80}.$$

Game G_2 (No random oracle collisions): $\mathcal{C}$ aborts if a collision occurs in the outputs of $\mathcal{H}_{\mathsf{scalar}}$ or $\mathcal{H}_{128}$. The probability is at most $q_{\mathrm{RO}}^2/2q + q_{\mathrm{RO}}^2/2^{129}$. Hence,

$$|\Pr[S_2] - \Pr[S_1]| \leq \frac{q_{\mathrm{RO}}^2}{2q} + \frac{q_{\mathrm{RO}}^2}{2^{129}}.$$

Game G_3 (Extraction from refresh proofs): For each valid refresh request, $\mathcal{C}$ applies the knowledge extractor for the BBS+ proof and the Schnorr equalities. By the forking lemma on the interactive version (challenge space $\mathbb{Z}_q$ for the BBS+ proof, and the Schnorr part), with probability at least $\epsilon^2/q_{\mathrm{RO}} - \epsilon/q$, $\mathcal{C}$ extracts the master secret k_{sub} and the BBS+ witness. If extraction fails, the proof is a forgery, which reduces to DL or q-SDH. The soundness error adds at most $\mathbf{Adv}^{\mathrm{DL}}(\mathcal{B}_2) + q_R/q$. Thus,

$$|\Pr[S_3] - \Pr[S_2]| \leq \mathbf{Adv}^{\mathrm{DL}}(\mathcal{B}_2) + \frac{q_R}{q}.$$

Game G_4 (Binding nullifiers to k_{sub}): $\mathcal{C}$ enforces the invariant that for any two accepted refresh requests with the same pk_C and epoch T, the extracted k_{sub} are equal. If not, we show a reduction to DL. Suppose $N_T = k \cdot H_T$ and $N'_T = k' \cdot H_T$ with $k \neq k'$. Then $(k - k') \cdot H_T = N_T - N'_T$. Given a DL challenge $H_T = g_1^a$, we can embed it and extract a from the known k, k', N_T, N'_T. The probability of this event is bounded by $\mathbf{Adv}^{\mathrm{DL}}(\mathcal{B}_3)$. Thus,

$$|\Pr[S_4] - \Pr[S_3]| \leq \mathbf{Adv}^{\mathrm{DL}}(\mathcal{B}_3).$$

Game G_5 (Simulation of issuance via BBS+ signing oracle): $\mathcal{C}$ replaces the honest issuance in $\mathcal{O}_{\mathsf{Issue}}$ with queries to a BBS+ signing oracle $\mathcal{O}_{\mathsf{Sign}}$. When $\mathcal{A}$ requests issuance for pk_C, $\mathcal{C}$ extracts k_{sub} from the proof of knowledge (standard Schnorr) and forwards the blinded commitment to $\mathcal{O}_{\mathsf{Sign}}$, which returns a signature on $(k_{\mathsf{sub}}, c_{\mathsf{max}}, E_{\mathsf{max}})$ under the secret key y (unknown to $\mathcal{C}$). The simulation is perfect, so $\Pr[S_5] = \Pr[S_4]$.

Game G_6 (Detection of master token forgeries): $\mathcal{C}$ checks if $\mathcal{A}$ outputs a valid master token for a pk_C^* that was never issued, or more tokens than q_{issue}. This directly breaks EUF-CMA of BBS+. We construct $\mathcal{B}_{\mathrm{SDH}}$ that simulates G_6 and outputs the forgery. Hence,

$$\Pr[\text{Case A win in } \mathsf{G}_6] \leq \mathbf{Adv}^{\text{q-SDH}}(\mathcal{B}_{\mathrm{SDH}}).$$

Game G_7 (Multiple daily tokens): $\mathcal{C}$ checks if $\mathcal{A}$ produced more than one daily token for the same pk_C^* and epoch T. By G_4, all such tokens have the same k_{sub} and thus identical nullifier $N_T = k_{\mathsf{sub}} \cdot H_T$. The server's $\mathcal{DB}_{\mathsf{epoch}}$ prevents duplicate nullifiers. To win, $\mathcal{A}$ must forge the freshness check, which is impossible in the simulation. The probability is bounded by the soundness error:

$$\Pr[\text{Case B win in } \mathsf{G}_7] \leq \frac{q_R}{q} + \mathrm{negl}(\lambda).$$

Game G_8 (Overspending): Suppose $\mathcal{A}$ attempts to overspend. Each spend proof includes a BEQ proof that the new balance $m \geq 0$ and Schnorr equalities ensuring $m = c_{\mathsf{bal}} - s$. By induction on the number of spends from a single daily token, the total spent cannot exceed c_{max}. To overspend, $\mathcal{A}$ must either (a) double-spend a token nullifier k_{cur} (prevented by $\mathcal{DB}_{\mathsf{spend}}$), (b) forge a spend proof (breaks DL), or (c) spend from multiple daily tokens derived from the same master token in the same epoch, which is Case B and already bounded. Thus,

$$\Pr[\text{Case C win in } \mathsf{G}_8] \leq \Pr[\text{Case B win}] + \mathbf{Adv}^{\mathrm{DL}}(\mathcal{B}_4).$$

Combining the games, we set $\mathcal{B}_{\mathrm{DL}}$ to be the adversary with maximum advantage among $\mathcal{B}_1, \ldots, \mathcal{B}_4$. The total advantage is bounded as stated. □

6.2 Unlinkability

Theorem 6.2 (Unlinkability)**.** Let $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ be a PPT adversary against the UNLINK game making at most q_{RO} random oracle queries, q_{refresh} challenge refresh queries, and q_{spend} challenge spend queries. Then there exists an adversary $\mathcal{B}_{\mathrm{SXDH}}$ such that

$$\mathbf{Adv}^{\mathrm{UNLINK}}(\mathcal{A}) \leq 2 \cdot \mathbf{Adv}^{\mathrm{SXDH}}(\mathcal{B}_{\mathrm{SXDH}}) + \frac{q_{\mathrm{RO}}^2}{q} + \frac{q_{\mathsf{refresh}} + q_{\mathsf{spend}}}{q} + \frac{1}{2^{128}} + \mathrm{negl}(\lambda).$$

Proof. We proceed via a sequence of games $\mathsf{H}_0, \ldots, \mathsf{H}_4$. Let $\Pr[S_{\mathsf{H}_i}]$ denote the probability that $\mathcal{A}$ outputs the correct bit b.

Game H_0: The real UNLINK game.

Game H_1 (Simulation of ZK proofs): $\mathcal{C}$ simulates all ZK proofs in the challenge phase. The difference is bounded by the ZK error:

$$|\Pr[S_{\mathsf{H}_1}] - \Pr[S_{\mathsf{H}_0}]| \leq \mathbf{Adv}^{\mathrm{DL}}(\mathcal{B}_{\mathrm{DL}}) + \frac{q_{\mathrm{RO}}^2}{2q} + 2^{-80}.$$

Game H_2 (Randomizing epoch nullifiers): For the challenge refresh query, $\mathcal{C}$ replaces $N_T = k_{\mathsf{sub}}^{(b)} \cdot H_T$ with a uniformly random $N_T \xleftarrow{\$} \mathbb{G}_1$ and simulates the refresh proof. We reduce to DDH in $\mathbb{G}_1$. Construct $\mathcal{B}_{\mathrm{SXDH}}$ that receives a DDH challenge $(g_1, A = g_1^a, B = g_1^b, C)$ where C is either g_1^{ab} or random. $\mathcal{B}_{\mathrm{SXDH}}$ sets $H_T = g_1$ and embeds a, b as the master secrets of the two clients. For the challenge, it sets $N_T = C$. If $C = g_1^{ab}$, then $N_T = k_{\mathsf{sub}}^{(b)} \cdot H_T$; otherwise it is random. The rest of the protocol is simulated using the ZK simulator. Note that the server is malicious, so its key is chosen by $\mathcal{A}$; $\mathcal{B}_{\mathrm{SXDH}}$ does not need to know the server's secret. Any distinguisher breaks DDH in $\mathbb{G}_1$, which is implied by SXDH. Hence,

$$|\Pr[S_{\mathsf{H}_2}] - \Pr[S_{\mathsf{H}_1}]| \leq \mathbf{Adv}^{\mathrm{SXDH}}(\mathcal{B}_{\mathrm{SXDH}}).$$

Game H_3 (Randomizing token commitments in spend): For challenge spend queries, $\mathcal{C}$ replaces the refund commitment K' with a commitment to a random balance (subject to the correct total spent) and simulates the spend proof. The BBS+ randomization ensures that A' and $\bar{A}$ are uniformly distributed among valid signatures, independent of the original token. The Pedersen commitment is perfectly hiding. Therefore, the view is statistically independent of b. The change is information-theoretic.

Game H_4 (Ideal game): $\mathcal{C}$ now ignores b entirely. The view of $\mathcal{A}$ is independent of b, so $\Pr[S_{\mathsf{H}_4}] = 1/2$.

Since H_3 and H_4 are identical up to statistical distance, $\Pr[S_{\mathsf{H}_3}] = 1/2$. Summing the bounds yields the theorem. □

6.3 Rate-Limit Soundness

Theorem 6.3 (Rate-Limit Soundness)**.** The EBUT construction satisfies rate-limit soundness under the Discrete Logarithm assumption. Concretely, for any PPT adversary $\mathcal{A}$,

$$\mathbf{Adv}^{\mathrm{RLS}}(\mathcal{A}) \leq \mathbf{Adv}^{\mathrm{DL}}(\mathcal{B}_{\mathrm{DL}}) + \frac{q_{\mathrm{RO}}^2}{2q} + \frac{q_{\mathsf{refresh}}}{q} + \frac{1}{2^{128}} + \mathrm{negl}(\lambda).$$

Proof. We analyze the two winning conditions of Game 3.

Case 1: Multiple daily tokens from the same master secret. Suppose $\mathcal{A}$ outputs $n \geq 2$ valid refresh tuples for epoch T with distinct nullifiers $N_T^{(i)} \neq N_T^{(j)}$ for some $i \neq j$, and all verifying under the same (adversary-chosen) server public key pk_S. The refresh proof's knowledge soundness guarantees that from each proof we can extract the master secret $k_{\mathsf{sub}}^{(i)}$ and the witness that $N_T^{(i)} = k_{\mathsf{sub}}^{(i)} \cdot H_T$ (up to blinding). By the forking lemma applied to the Schnorr equality (challenge space 2^{128}), extraction succeeds with probability at least $\epsilon^2/q_{\mathrm{RO}} - \epsilon/2^{128}$, where ϵ is the success probability of the proof. If extraction fails, the adversary has forged the proof, reducing to DL.

Now, if there exist $i \neq j$ with $k_{\mathsf{sub}}^{(i)} = k_{\mathsf{sub}}^{(j)}$, then $N_T^{(i)} = k \cdot H_T = N_T^{(j)}$, contradicting the distinctness of nullifiers. Hence this case cannot occur unless the adversary breaks the binding of the equality proof, which reduces to DL. The probability of this event is bounded by $\mathbf{Adv}^{\mathrm{DL}}(\mathcal{B}_{\mathrm{DL}}^{(1)})$.

Case 2: Overspending. Suppose $\mathcal{A}$ produces a set of spend transactions within epoch T that collectively consume more than c_{max} credits from a single master token without double-spending the same nullifier. Each spend transaction includes a spend proof that reduces to a daily token

with balance c_{max}. The spend proof's BEQ range proof ensures $m \geq 0$, and the Schnorr equalities enforce $m = c_{\mathsf{bal}} - s$. By induction on the number of spends, the total spent cannot exceed c_{max}. To overspend, $\mathcal{A}$ must either (a) double-spend a token nullifier (prevented by $\mathcal{DB}_{\mathsf{spend}}$; any collision would imply a forgery of the spend proof), (b) produce a spend proof with an invalid balance (breaks soundness, reduces to DL), or (c) spend from multiple daily tokens derived from the same master secret in the same epoch, which is Case 1 and already bounded.

Thus, the overall advantage is bounded as stated. □

7 Implementation Guidance

We now provide detailed, production-oriented guidance for implementing the EBUT protocol. These recommendations are essential for security, interoperability, and performance.

7.1 BBS+ Library

Implementers should use a CFRG-compliant BBS+ library that supports blind issuance and zero-knowledge proofs of possession. The proof described in Protocol 1 of the ACT paper uses multiplicative randomization ($A' = A^{r_1}$) and is proven secure under the assumptions of this paper. Ensure that the verification equations match exactly those specified in Section 4. In particular, the pairing check must be $e(A', \mathsf{pk}_S) \stackrel{?}{=} e(\bar{A}, g_2)$ after proper randomization.

7.2 Cross-Curve Batched Equality Proof Implementation

The EBUT protocol uses the BEQ to prove that a value m is in $[0, 2^{32} - 1]$ and is consistent with the BLS12-381 commitment used in the BBS+ proof. To implement this correctly:

1. **BLS12-381 Commitment:** Compute C_{BLS} using the BBS+ generators h_4 (value) and h_0 (blinding) as specified in Sections 5 and 6.

2. **Ristretto Commitment:** Independently, using the `curve25519-dalek` library with its **standard Ristretto generators** `RISTRETTO_BASEPOINT_POINT` (for G) and `RISTRETTO_BASEPOINT_COMPRESSED` (for H, derived via hash-to-curve with domain separation), compute $C_{25519} = m \cdot G + r_{25519} \cdot H$ where $r_{25519} \stackrel{\$}{\leftarrow} \mathbb{Z}_{q_{25519}}$.

3. **BEQ Proof Generation:** Execute the BEQ prover algorithm (Section 5) with inputs $(m, r_{\text{BLS}}, r_{25519})$, context $\text{ctx} = H_{\text{ctx}}$, and auxiliary commitments being the BBS+ proof components ($A', \bar{A}, T_{\mathsf{BBS}}$, etc.). The prover produces $\pi_{\text{BEQ}} = (C_{25519}, R_{\text{BLS}}, R_{25519}, z, z_{\text{BLS}}, z_{25519}, \pi_{\text{range}})$.

4. **Verification:** The server verifies π_{BEQ} against C_{BLS} and the auxiliary commitments. The Dalek Bulletproof verification uses the library's **default generators**.

Critical: Do **not** attempt to override the Dalek Bulletproof generators with BLS12-381 points. The binding between curves is established by the Sigma protocol equality check, not by generator substitution. The Bulletproof transcript must include the BBS+ commitments to prevent splicing attacks.

7.3 Epoch Nullifier Computation

The client computes the epoch base point deterministically:

$$H_T = \mathcal{H}_{\mathbb{G}_1}(\text{“EBUT:Epoch:”} \parallel T) \in \mathbb{G}_1.$$

The hash function $\mathcal{H}_{\mathbb{G}_1}$ should be instantiated using a standard hash-to-curve method (e.g., RFC 9380) with domain separation tag `"EBUT:Epoch:"`. The epoch T is encoded as a 32-bit big-endian integer. The nullifier is $N_T = k_{\mathsf{sub}} \cdot H_T$. The server **must not** use a secret-derived base; the trustless nullifier property relies on the public derivation of H_T.

7.4 Idempotency and Torn Transaction Recovery

Network failures and client crashes can lead to retried requests. To prevent double-spend errors and improve user experience, the server must implement idempotency caches.

Refresh Phase Idempotency. When the server issues a Daily Token, it computes a cache key using a length-prefixed encoding to prevent byte-shifting collisions:

$$\mathtt{key}_{\mathsf{ref}} = \mathcal{H}_{\mathsf{scalar}}(\text{“EBUT:Idempotency:Refresh”} \parallel \mathsf{len}(\pi_{\mathsf{refresh}}) \parallel \pi_{\mathsf{refresh}}),$$

where $\mathsf{len}(\cdot)$ denotes a fixed-width (e.g., 4-byte big-endian) encoding of the byte length of the following field. This length-prefix prevents an adversary from shifting bytes between fields to produce the same concatenated input for different payloads. The mapping $\mathtt{key}_{\mathsf{ref}} \mapsto (A_{\mathsf{daily}}, e_{\mathsf{daily}}, s'_{\mathsf{daily}})$ is stored for the remainder of the epoch (or at least several minutes). If a client resubmits an identical π_{refresh}, the server returns the cached response **without** re-executing the expensive verification (pairings, MSMs) and **without** re-checking the epoch nullifier in $\mathcal{DB}_{\mathsf{epoch}}$.

Spend Phase Idempotency. For spend requests, the cache key uses the same length-prefixed encoding:

$$h_{\mathsf{id}} = \mathcal{H}_{\mathsf{scalar}}(\text{“EBUT:Idempotency:Spend”} \parallel \mathsf{len}(\pi_{\mathsf{spend}}) \parallel \pi_{\mathsf{spend}} \parallel \eta).$$

The fixed length of η (16 bytes) eliminates ambiguity on that side; the length prefix on π_{spend} prevents byte-shifting between the proof and the nonce field. Raw concatenation $\pi_{\mathsf{spend}} \parallel \eta$ without a length delimiter is **prohibited** because a byte shifted from the end of π_{spend} into η yields an identical octet string, mapping two distinct requests to the same cache key. The server stores $h_{\mathsf{id}} \mapsto (A^*, e^*, s'^*)$ for the duration of the epoch plus the grace period—the same TTL assigned to the corresponding k_{cur} entry in $\mathcal{DB}_{\mathsf{spend}}$. **This TTL must exactly match or exceed the TTL of the token nullifier.** If the idempotency cache entry expires before k_{cur} expires in $\mathcal{DB}_{\mathsf{spend}}$, a client whose network dropped after the server committed the transaction but before the client received the response will later retry, find no cache hit, reach the double-spend check, and be permanently rejected with a 409 Conflict—losing their token with no recourse. **Crucially, the idempotency cache must be checked *before* the double-spend database.** If a match is found, the cached refund token is returned immediately. This prevents legitimate retries from being incorrectly flagged as double spends.

7.5 API Payload Specification

Implementers must transmit the public statements alongside the proof. We provide JSON schemas for both refresh and spend requests. All binary data is encoded in hexadecimal.

7.5.1 Refresh Request Payload

```
{
  "T": <32-bit unsigned integer>,
  "N_T": <48-byte compressed G1 point, hex>,
  "K_daily": <48-byte compressed G1 point, hex>,
  "C_Delta": <48-byte compressed G1 point, hex>,
  "pi_refresh": {
    "A_prime": <48-byte compressed G1 point, hex>,
    "A_bar": <48-byte compressed G1 point, hex>,
    "T_BBS": <48-byte compressed G1 point, hex>,
    "T_scale_N": <48-byte compressed G1 point, hex>,
    "T_N": <48-byte compressed G1 point, hex>,
    "T_scale_D": <48-byte compressed G1 point, hex>,
    "T_D": <48-byte compressed G1 point, hex>,
    "T_scale_B": <48-byte compressed G1 point, hex>,
    "T_B": <48-byte compressed G1 point, hex>,
    "z_e": <32-byte scalar, hex>,
    "z_r1": <32-byte scalar, hex>,
    "z_s_tilde": <32-byte scalar, hex>,
    "z_k_tilde": <32-byte scalar, hex>,
    "z_c_tilde": <32-byte scalar, hex>,
    "z_E_tilde": <32-byte scalar, hex>,
    "z_u": <32-byte scalar, hex>,
    "z_v": <32-byte scalar, hex>,
    "z_w": <32-byte scalar, hex>,
    "pi_BEQ_exp": <binary blob, base64 or hex>
  }
}
```

7.5.2 Spend Request Payload

```
{
  "s": <32-bit unsigned integer>,
  "T_issue": <32-bit unsigned integer>,
  "k_cur": <32-byte scalar, hex>,
  "eta": <16-byte nonce, hex>,
  "eta_proof": <optional Merkle inclusion proof for high-privacy mode>,
  "K_prime": <48-byte compressed G1 point, hex>,
  "C_BP": <48-byte compressed G1 point, hex>,
  "pi_spend": {
    "A_prime": <48-byte compressed G1 point, hex>,
    "A_bar": <48-byte compressed G1 point, hex>,
    "T_BBS": <48-byte compressed G1 point, hex>,
    "T_scale_T": <48-byte compressed G1 point, hex>,
    "T_total": <48-byte compressed G1 point, hex>,
    "T_scale_R": <48-byte compressed G1 point, hex>,
    "T_refund": <48-byte compressed G1 point, hex>,
```

```
    "T_scale_BP": <48-byte compressed G1 point, hex>,
    "T_BP": <48-byte compressed G1 point, hex>,
    "z_e": <32-byte scalar, hex>,
    "z_r1": <32-byte scalar, hex>,
    "z_s_tilde": <32-byte scalar, hex>,
    "z_c_tilde": <32-byte scalar, hex>,
    "z_u": <32-byte scalar, hex>,
    "z_v": <32-byte scalar, hex>,
    "z_w": <32-byte scalar, hex>,
    "pi_BEQ_spend": <binary blob, base64 or hex>
  }
}
```

7.6 Serialization and Domain Separation

- **Group Elements:** All $\mathbb{G}_1$ elements must be serialized in **compressed** form (48 bytes for BLS12-381). Deserialization must validate that the point lies in the prime-order subgroup ($q \cdot P = 1_{\mathbb{G}_1}$) and is canonically encoded.
- **Scalars:** All scalars must be serialized as **fixed-width 32-byte big-endian** integers. During deserialization, any 32-byte scalar x must be rejected if $x \geq q$ (the group order). This prevents scalar malleability attacks.
- **Domain Separation:** All hash function invocations must be prefixed with a domain separation string (e.g., `"EBUT:Refresh:v1"`). The context hash H_{ctx} must be included as the first element in every Fiat–Shamir challenge.
- **Fiat–Shamir Hash Width:** To prevent modulo bias, every use of $\mathcal{H}_{\mathsf{scalar}}$ for Fiat–Shamir challenges **must** produce an output wider than the field order before reduction. Concretely: use SHA-512 (or BLAKE2b-512) yielding 64 bytes, or use RFC 9380 `expand_message_xmd` with sufficient output length, then reduce the wide integer modulo q. Implementing $\mathcal{H}_{\mathsf{scalar}}$ as `SHA-256(data) mod q` is **prohibited**.
- **Subgroup Checks:** Every received $\mathbb{G}_1$ element must be validated to lie in the prime-order subgroup to prevent small-subgroup attacks.

7.7 Nullifier Database Architecture

The server maintains two logical sets:

- $\mathcal{DB}_{\mathsf{epoch}}$: stores compressed $\mathbb{G}_1$ points (48 bytes). Entries are assigned a TTL of *epoch duration + grace period* at insertion time. **The database must not be bulk-cleared at the epoch boundary**; individual entries expire after their TTL elapses.
- $\mathcal{DB}_{\mathsf{spend}}$: stores canonical scalars (32 bytes). Entries are assigned a TTL of *epoch duration + grace period* at insertion time. **The database must not be bulk-cleared at the epoch boundary**; individual entries expire after their TTL elapses.
- **Nonce store** ($\mathcal{N}_{\mathsf{spent}}$): stores the 16-byte session nonces η that have been consumed during spend operations. Each entry **must** be assigned a TTL equal to *epoch duration + grace period*—the same TTL as $\mathcal{DB}_{\mathsf{spend}}$. Omitting a TTL causes permanent accumulation: at

high spend velocity, uncontrolled growth of the nonce set will exhaust Redis memory. In Redis, store each nonce as `SET "nonce:{eta_hex}" 1 EXAT {epoch_end + grace_seconds}` to guarantee per-entry expiry independent of other entries.

Epoch Grace Period. Critical invariant: the grace period and database expiry must be consistent. If the server accepts refresh or spend requests for epoch $T-1$ during a grace window of duration Δ_{grace} after the start of epoch T, then entries written to $\mathcal{DB}_{\mathsf{epoch}}$ and $\mathcal{DB}_{\mathsf{spend}}$ during epoch $T-1$ *must remain readable* for at least Δ_{grace} into epoch T. Bulk-wiping either database at the epoch boundary while simultaneously accepting grace-period requests creates an exploit window in which an attacker may re-use an epoch-$T-1$ nullifier that appears fresh because its record was erased. Concretely, use per-key TTL-based storage: in Redis, store each nullifier as an individual key (e.g., `SET "epoch:nullifier:{N_T}" 1 EXAT {epoch_end + grace_seconds}`) so that each entry expires only after the grace window has fully closed, independent of other entries.

Atomicity. All insertions must be conditional and atomic. In SQL, use `INSERT ... ON CONFLICT DO NOTHING`. In Redis, use `SETNX`. If an insertion fails due to a duplicate key, the request must be rejected immediately.

High-Velocity Spends. A user may legitimately generate up to c_{max} spends in a short period. To handle this efficiently, implementers **must** use an exact, authoritative data structure as the definitive store for $\mathcal{DB}_{\mathsf{spend}}$ and the nonce spent-set. In Redis, store each nullifier as an individual key with a per-key TTL (e.g., `SET "spend:nullifier:{k_cur}" 1 EX {ttl_seconds}`), which provides $O(1)$ membership tests and per-entry expiry without affecting other entries. A probabilistic structure such as a Bloom filter **must not** serve as the authoritative store: as the filter fills during an epoch, the false-positive rate rises, causing legitimate spends to be incorrectly rejected. A Bloom filter or Cuckoo filter *may* be placed in front of the exact store as a fast-reject optimisation to avoid unnecessary database lookups for obviously duplicate entries, but any positive result from the filter must be confirmed against the authoritative store before the request is rejected.

7.8 Verification Order (Optimized)

To prevent CPU exhaustion attacks from malformed proofs, the server must perform checks in the following order (fastest to slowest). **All database writes (state mutations) are deferred to the final step and executed only after all cryptographic checks pass.** Performing writes earlier exposes the server to denial-of-service attacks in which an adversary submits a valid nonce with garbage cryptographic proofs, permanently burning the legitimate user's nonce before the proof is validated.

1. Epoch and parameter bounds (e.g., $s > 0$, T valid).
2. **Idempotency cache check** (return cached token if hit).
3. Nonce read-only check (Merkle proof if applicable; **no write**).
4. Nullifier read-only check and zero-scalar rejection ($N_T \neq 1_{\mathbb{G}_1}$, $k_{\mathsf{cur}} \neq 0$); **no write**.
5. Basic inequality checks ($A' \neq 1_{\mathbb{G}_1}$, $T_{\mathsf{BBS}} \neq 1_{\mathbb{G}_1}$).
6. Bridge equation checks (additive group operations, fast).

7. **Schnorr MSM verification** (BBS+ core without pairing) – ≈ 0.5 ms.
8. **Bulletproof verification** (or BEQ verification) – $\approx$ 1–2 ms.
9. **Bilinear pairing checks** – $\approx$ 2–4 ms.
10. **Atomic state mutation** (insert nullifier into $\mathcal{DB}_{\mathsf{spend}}$, mark nonce as spent; reject on concurrent duplicate).

7.9 Edge Cases

- $s = 0$**:** Reject explicitly. Spending zero credits is meaningless.
- **Zero scalars:** The client must ensure $k_{\mathsf{sub}} \neq 0$, $k_{\mathsf{daily}} \neq 0$, $k^* \neq 0$. The server must reject any nullifier $k_{\mathsf{cur}} = 0$ or $N_T = 1_{\mathbb{G}_1}$.
- **Zero balance:** A token with $c_{\mathsf{bal}} = 0$ is allowed; further spends will fail because $s > 0$ and $m \geq 0$ cannot be satisfied.
- **Clock synchronization:** Accept refresh requests with T within a small tolerance (e.g., ± 1 epoch) to accommodate client clock drift, but enforce the expiry check strictly.
- c_{max} **range:** The server must enforce $c_{\mathsf{max}} \in [0, 2^{32} - 1]$ during minting to ensure compatibility with 32-bit range proofs.

7.10 Privacy and Operational Considerations

Transport-Layer Anonymity. The standard nonce acquisition method (`GET /nonce`) creates a temporal correlation between the client's IP address and the subsequent spend request, breaking anonymity at the network layer. For deployments where transport-layer privacy is critical, implementers **must** use the high-privacy nonce distribution method: the server periodically publishes a Merkle root of pre-generated nonces. The client selects a nonce, obtains its Merkle inclusion proof, and submits the proof with the spend request. The server verifies the proof and, once all cryptographic checks pass, marks the nonce as spent in an **exact, authoritative store** (e.g., a Redis `SADD` with TTL-based expiry). This decouples nonce acquisition from spend submission. A probabilistic structure such as a Bloom filter **must not** be used as the authoritative record of spent nonces: its false-positive rate rises as the epoch progresses, causing legitimate spends to be incorrectly rejected with HTTP 409 Conflict responses. A Bloom filter may be used as a fast-reject layer in front of the authoritative store, but any positive result must be confirmed with an exact lookup before the request is rejected.

Epoch Boundary Burst Capacity. The 5-minute grace period allows a user to hold an unexpired token from the previous epoch while also minting a new token for the current epoch. During this overlap window, the user can spend up to $2 \cdot c_{\mathsf{max}}$ credits. This does not violate the long-term rate limit (one token per epoch), but it does permit a brief localized burst. This trade-off is considered acceptable for fault tolerance. If strict per-minute rate limiting is required, it must be enforced by an external system.

Cross-Epoch Linkability Assumption. The unlinkability of N_T across epochs relies on the SXDH assumption in Type-3 pairings like BLS12-381. This provides computational (rather than information-theoretic) secrecy, which is the standard security guarantee for such protocols.

Token Theft Resistance. Even if an attacker intercepts a `SpendRequest` and submits it before the victim, the attacker receives only the blind signature (A^*, e^*, s'^*). Without the local blinding factor r^*, the attacker cannot compute the final token secret $s^* = r^* + s'^*$. The token is cryptographically useless to the attacker, while the victim can safely resubmit (or receive from the idempotency cache) and unblind correctly.

Plaintext Spend Amount and Statistical Fingerprinting. The spend amount s is transmitted in plaintext (Section 4.4.3, Step 1). While the token nullifiers k_{cur} are perfectly randomized and unlinkable across spend requests, the metadata value s creates a potential behavioral side-channel. A server that logs spend requests may perform statistical fingerprinting on the sequence of s values to link intra-epoch transactions originating from the same user. For most deployments this is an acceptable trade-off, as the server is already trusted not to attempt de-anonymization via timing or traffic analysis. If strict intra-epoch unlinkability of spend amounts is required (e.g., in high-assurance deployments), the amount s must be committed and hidden, with the server verifying $\sum s_i \leq c_{\mathsf{max}}$ over the epoch using a homomorphic accumulator rather than plaintext step-downs. This extension is left for future work.

8 Performance Evaluation

We provide estimates for the EBUT protocol on BLS12-381 and Curve25519. The cross-curve BEQ proof size is approximately 672 bytes for the Bulletproof plus 160 bytes for the Sigma protocol responses, totaling under 1 KB. The overall proof sizes are similar to the ACT protocol.

Phase	**Prover (Client)**	**Verifier (Server)**	**Proof Size**
Master Mint	2 exp + PoK	2 exp	$\approx$ 128 B
Epoch Refresh	BBS+ proof + BEQ	2 pairings + BEQ	$\approx$ 3.0 KB
Spend	BBS+ proof + BEQ + equalities	2 pairings + BEQ	$\approx$ 3.2 KB

Table 1: Performance estimates for EBUT.

9 Conclusion

We have presented Epoch-Bound Usage Tokens (EBUT), a primitive that enforces rate-limited anonymous access by cryptographically binding tokens to discrete time epochs. Our construction combines blind BBS+ signatures, Pedersen commitments, and zero-knowledge proofs to achieve strong privacy guarantees while preventing token accumulation and burst consumption. We formalized the security properties of EBUT—unforgeability, unlinkability, and rate-limit soundness—and provided rigorous, game-based proofs under standard assumptions. The integration of cross-curve batched equality proofs ensures practical efficiency without compromising security. We provided a complete, production-ready specification with detailed implementation guidance, API schemas, and performance estimates. EBUT offers a robust solution for privacy-preserving subscription services and rate-limited resource access in anonymous systems.

References

[1] Anonymous Credit Tokens (ACT): A Complete Formal Specification. *Cryptology ePrint Archive*, 2024.

[2] F. B. Durak, A. Talayhan, L. Marco, and S. Vaudenay. Non-Transferable Anonymous Tokens by Secret Binding. *Full version, May 8th, 2024*.

[3] J. Camenisch and A. Lysyanskaya. An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation. *EUROCRYPT 2001*.

[4] J. Camenisch and A. Lysyanskaya. A Signature Scheme with Efficient Protocols. *SCN 2002*.

[5] M. H. Au, W. Susilo, and Y. Mu. Constant-Size Dynamic k-TAA. *SCN 2006*.

[6] M. Lodder, T. Looker, and A. Whitehead. The BBS Signature Scheme. *IETF Internet-Draft draft-irtf-cfrg-bbs-signatures-05*, 2024.

[7] D. Boneh and X. Boyen. Short Signatures Without Random Oracles. *EUROCRYPT 2004*.

[8] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. Bulletproofs: Short Proofs for Confidential Transactions and More. *IEEE S&P 2018*.

[9] D. Pointcheval and J. Stern. Security Arguments for Digital Signatures and Blind Signatures. *Journal of Cryptology*, 13(3):361–396, 2000.

[10] D. Boneh and V. Shoup. A Graduate Course in Applied Cryptography. *Version 0.6*, 2023.